

Mount Kenya University

Student Name: IMMACULATE MUTHEU MULI

Admission Number: BSCCS/2024/33678

Course/Class: COMPUTER SCIENCE JANUARY 2024

Unit Code: BIT4133

Unit Name: NATURAL LANGUAGE PROCESSING

Lecturer Name: MR. MICHAEL NYORO

Semester/Academic Year: FOURTH YEAR

NLP Project Title: CBK Report Analyser

Technologies Used

- Python
- NLTK
- spaCy
- Gensim (Word2Vec)
- pdfplumber
- pandas
- matplotlib
- Google Colab

Google Colab Link

<https://colab.research.google.com/drive/1n7Lh1C7h75jyABjd-qtnPnyS8uX7SXOA?usp=sharing>

https://colab.research.google.com/drive/1yTooPvMbwfJq1NrN_yEY0lReaXORqUKV?usp=sharing

Github Link

https://github.com/immaculatemuli/Natural_Language_Processing.git

Table of Contents

Week 1: Introduction to NLP and Applications	5
Fig 1: Python Installation.....	6
Fig 2: Google Colab	6
Fig 3: NLTK Installation.....	7
Fig 4: Tokenization Output	7
Fig 5: Stopwords Removal Output	8
Fig 6: NLP Application Research Evidence.....	8
Week 2: N-gram Models and POS Tagging	10
Fig 1: Bigram or Trigram output	10
Fig 2: Python Code for POS Tagging	10
Fig 3: POS Tagging Example.....	11
Fig 4: Language Prediction Example and Output	11
Fig 5: Sentence Analysis Output	12
Week 3: Hidden Markov Models and Sequence Labeling	13
Fig 1: Sequence Labeling code	13
Fig 2: Hidden Markov ModelOutput	13
Fig 3: Named Entity Recognition Example and Output.....	14
Fig 4: Sentence Labeling Results	15
Fig 5: Dataset Used for Training or Testing.....	16
Week 4: Syntactic And Semantic Analysis.....	17
PRACTICAL TASK 1	17
Fig 1: Dependency Parsing Using spaCy.....	17
Fig 2: Semantic Similarity	18
CLASS ASSIGNMENT 1.....	18
Fig 1: Dependency Parsing Using spaCy	19
Fig 2: Semantic Similarity	19
Week 5: Mini NLP Project	21

Deliverable 1: Source code	21
Deliverable 2: Screenshots of chatbot interaction.....	21
Deliverable 3: Short Explanation Document	22
Deliverable 4: Reflection on challenges faced.....	23
Week 6: Word Embeddings and Distributed Representations	23
CLASS DEMONSTRATION	23
Fig 1:Installing Gensim	23
Fig 2:Training a Small Word2Vec Model	24
Fig 3:Finding Similar Words.....	24
Fig 4:Word Similarity Score.....	25
Fig 5:Visualizing Word Embeddings	25
CLASS ACTIVITY	26
Fig 1:Comparing Representation Methods.....	26
Fig 2:Similarity Investigation	27
Fig 3:Real-World NLP Discussion	27
PRACTICAL TASK.....	28
Fig 1:Train a Word2Vec Model	28
Fig 2:Similarity Analysis	28
Fig 3:Comparison Report	29
ADVANCED PRACTICAL TASK.....	29
Fig 1:Generate embeddings.....	29
Fig 2:Analyze semantic relationships	29
Fig 3:Visualize word clusters (PCA and t-SNE).....	30
Document Observations	31
CHALLENGE TASK - Modern Embedding Approaches	31
Week 7: Neural Language Models and Deep Learning for NLP	34
CLASS DEMONSTRATION	34
Fig 1: Installing TensorFlow.....	34
Fig 2: Building a Simple Neural Network.....	34

Fig 3: Understanding the Layers	34
Fig 4: Text Tokenization	35
Fig 5: Converting Text to Sequences.....	35
Fig 6: Simple Prediction Example	36
CLASS ACTIVITY	36
Fig 1: Language Model Comparison	36
Fig 2: Neural Network Components	37
Fig 3: NLP Application Discussion	37
PRACTICAL TASK	38
Fig 1: Text Prediction System	38
Fig 2: TensorFlow NLP Exercise	39
Fig 3: The Mini Predictive Text Application.....	40
Week 8: Transformer Models and Attention Mechanisms	41
CLASS EXERCISE 1	41
CLASS EXERCISE 2.....	41
PRACTICAL TASK 1	42
Fig 1: Hugging Face Transformers that performs sentiment analysis.....	42
PRACTICAL TASK 2	43
Fig 2: Text-generation application using GPT-2	43
EXTRA TASKS	44
Week 1: Extra tasks	44
Fig 1: Stemming and lemmatization.....	44
Fig 2: Word frequency and counter visualization	44
Fig 3: NLU vs NLG demonstration	45
Week 2: Extra tasks	46
Fig 1: N-gram Frequency Chart	46
Week 3: Extra tasks	47
Fig 1: displaCy NER Visualization	47
Fig 2: Real HMM trained on CBK Report data	47

Week 1: Introduction to NLP and Applications

Title: Introduction to NLP Concepts and Text Processing

Fig 1: Python Installation

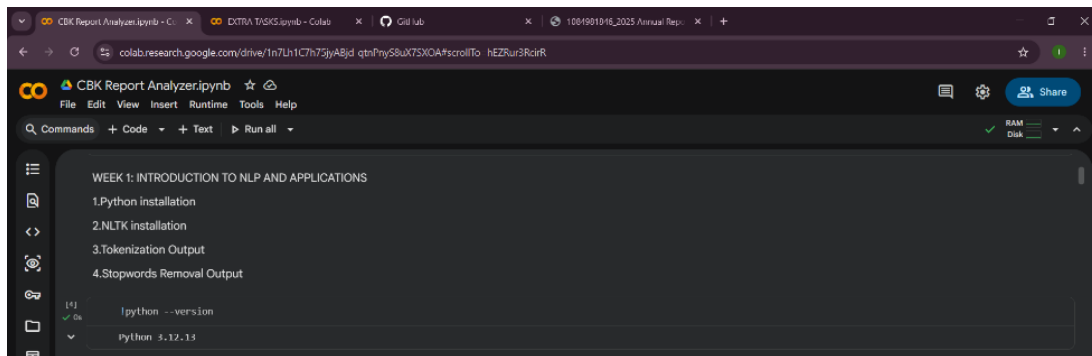


Fig 2: Google Colab

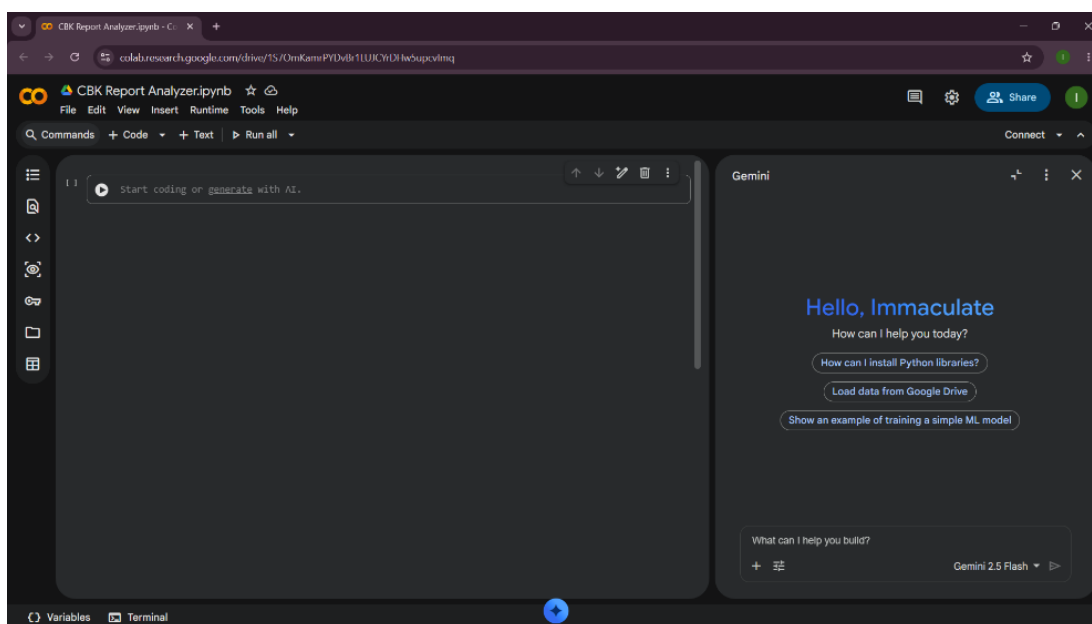
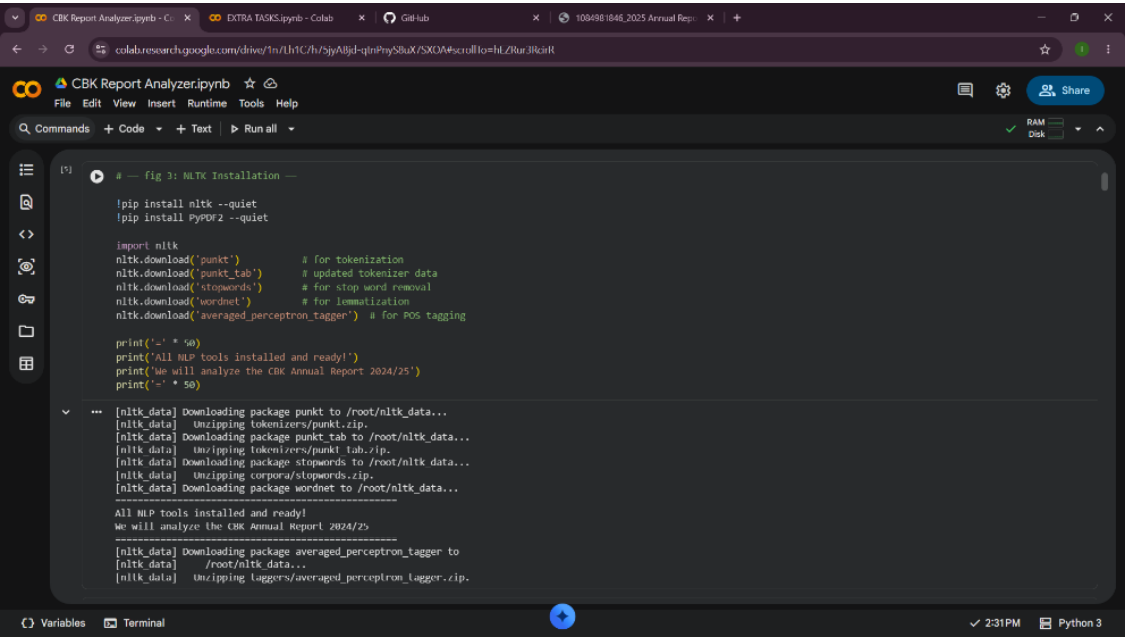


Fig 3: NLTK Installation



The screenshot shows a Jupyter Notebook titled "CBK Report Analyzer.ipynb" in a Google Colab environment. The notebook is in the "Code" view, and the code cell is titled "# fig 3: NLTK Installation". The code installs NLTK and its dependencies, including punkt, punkt_tab, stopwords, wordnet, and averaged_perceptron_tagger. The output shows the progress of downloading and unzipping these packages.

```
# fig 3: NLTK Installation --

!pip install nltk --quiet
!pip install PyPDF2 --quiet

import nltk
nltk.download('punkt') # for tokenization
nltk.download('punkt_tab') # updated tokenizer data
nltk.download('stopwords') # for stop word removal
nltk.download('wordnet') # for lemmatization
nltk.download('averaged_perceptron_tagger') # for POS tagging

print('-' * 50)
print('All NLP tools installed and ready!')
print('We will analyze the CBK Annual Report 2024/25')
print('-' * 50)

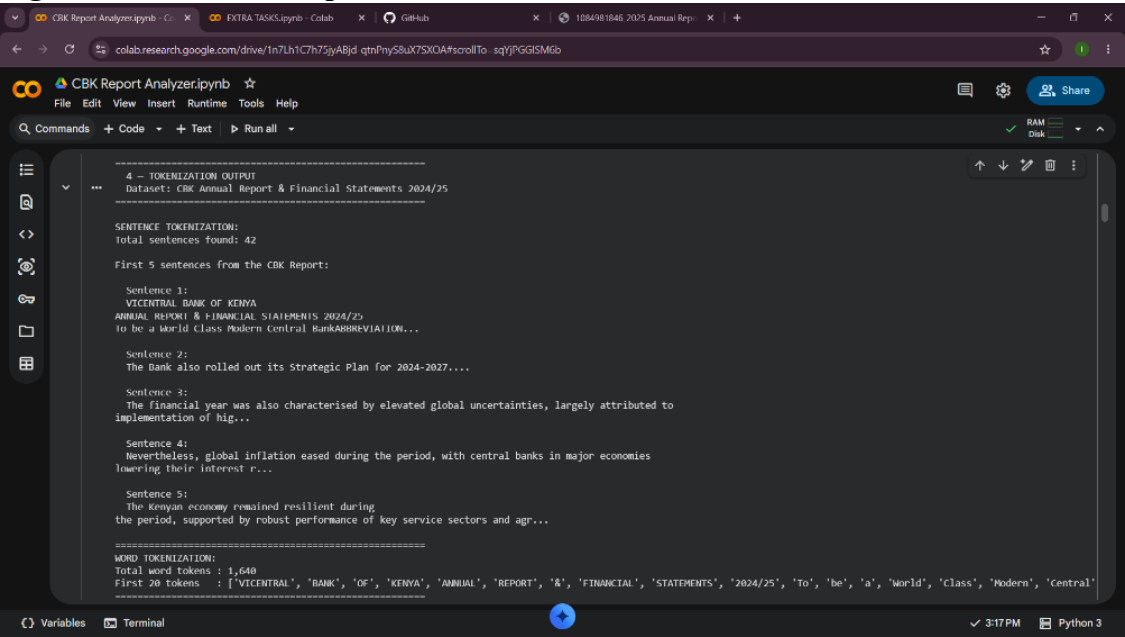
...

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Unzipping corpora/wordnet.zip.

All NLP tools installed and ready!
We will analyze the CBK Annual Report 2024/25

[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger.zip.
```

Fig 4: Tokenization Output



The screenshot shows the same Jupyter Notebook, now displaying the output of the tokenization process. The output is divided into two sections: "SENTENCE TOKENIZATION" and "WORD TOKENIZATION". The sentence tokenization output shows the total number of sentences found (42) and the first 5 sentences from the report. The word tokenization output shows the total number of word tokens (1,640) and the first 20 tokens, which are listed as: ['VICENTRAL', 'BANK', 'OF', 'KENYA', 'ANNUAL', 'REPORT', '&', 'FINANCIAL', 'STATEMENTS', '2024/25', 'to', 'be', 'a', 'World', 'Class', 'Modern', 'Central'].

```
4 - TOKENIZATION OUTPUT
Dataset: CBK Annual Report & Financial Statements 2024/25

=====

SENTENCE TOKENIZATION:
Total sentences found: 42

First 5 sentences from the CBK Report:

Sentence 1:
VICENTRAL BANK OF KENYA
ANNUAL REPORT & FINANCIAL STATEMENTS 2024/25
to be a World Class Modern Central BankABBREVIATION...

Sentence 2:
The Bank also rolled out its Strategic Plan for 2024-2027....

Sentence 3:
The financial year was also characterised by elevated global uncertainties, largely attributed to
implementation of HIG...

Sentence 4:
Nevertheless, global inflation eased during the period, with central banks in major economies
lowering their interest r...

Sentence 5:
The Kenyan economy remained resilient during
the period, supported by robust performance of key service sectors and agr...

=====

WORD TOKENIZATION:
Total word tokens : 1,640
First 20 tokens : ['VICENTRAL', 'BANK', 'OF', 'KENYA', 'ANNUAL', 'REPORT', '&', 'FINANCIAL', 'STATEMENTS', '2024/25', 'to', 'be', 'a', 'World', 'Class', 'Modern', 'Central']
```

Fig 5: Stopwords Removal Output

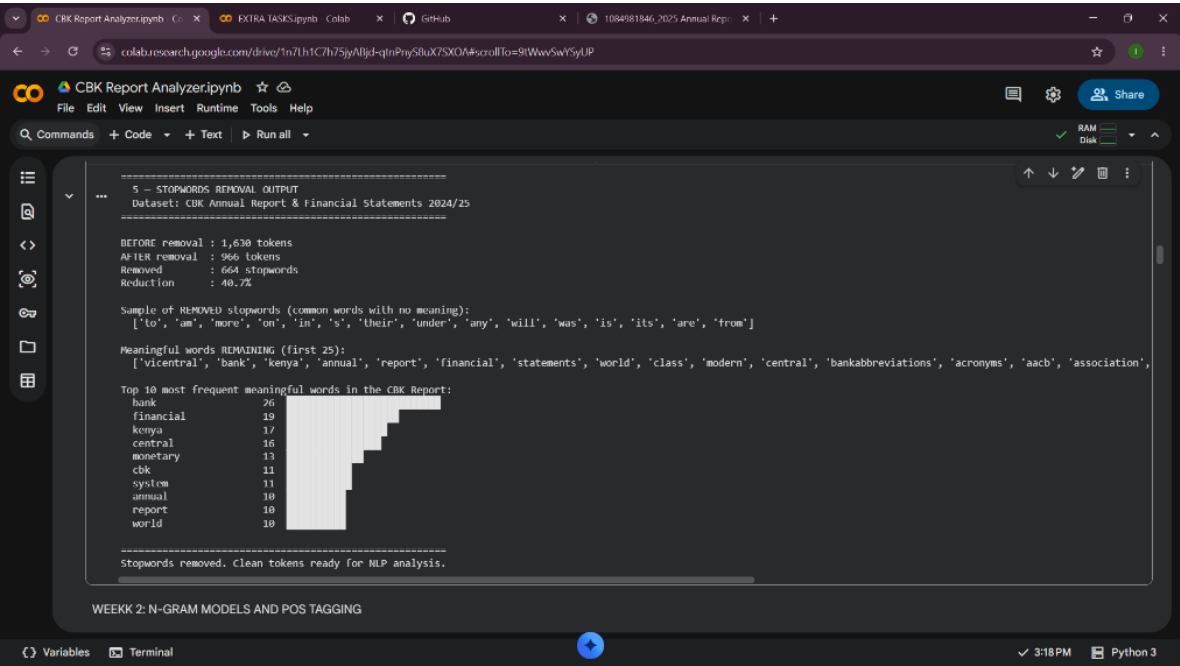
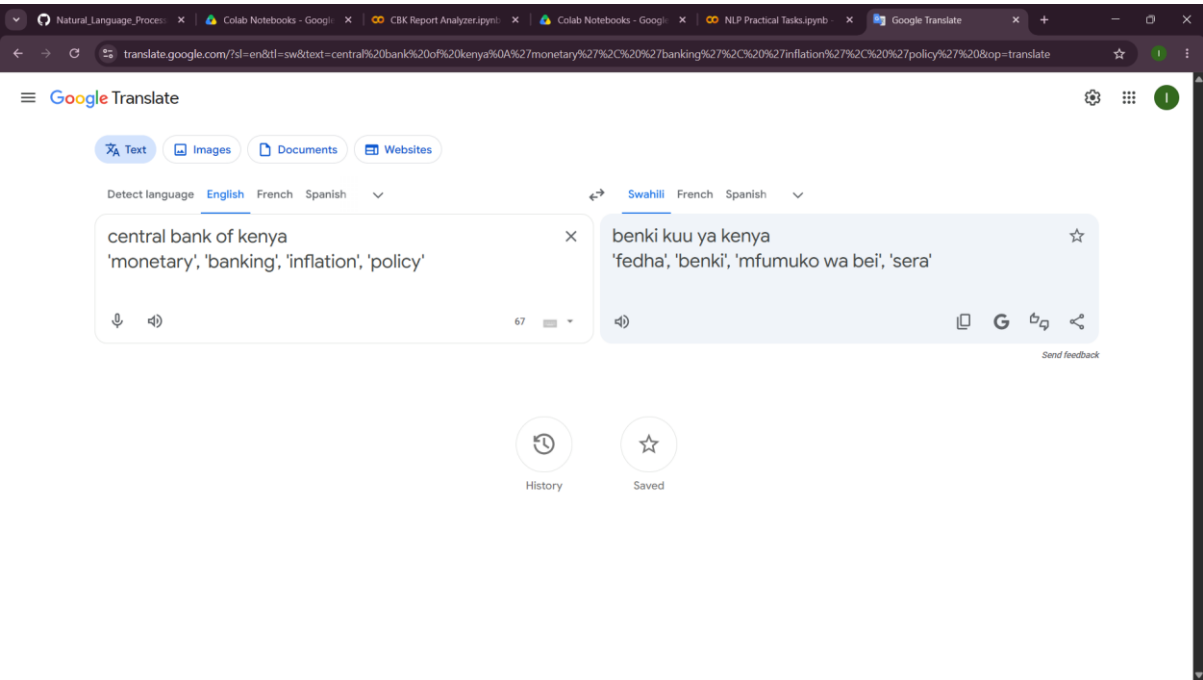


Fig 6: NLP Application Research Evidence



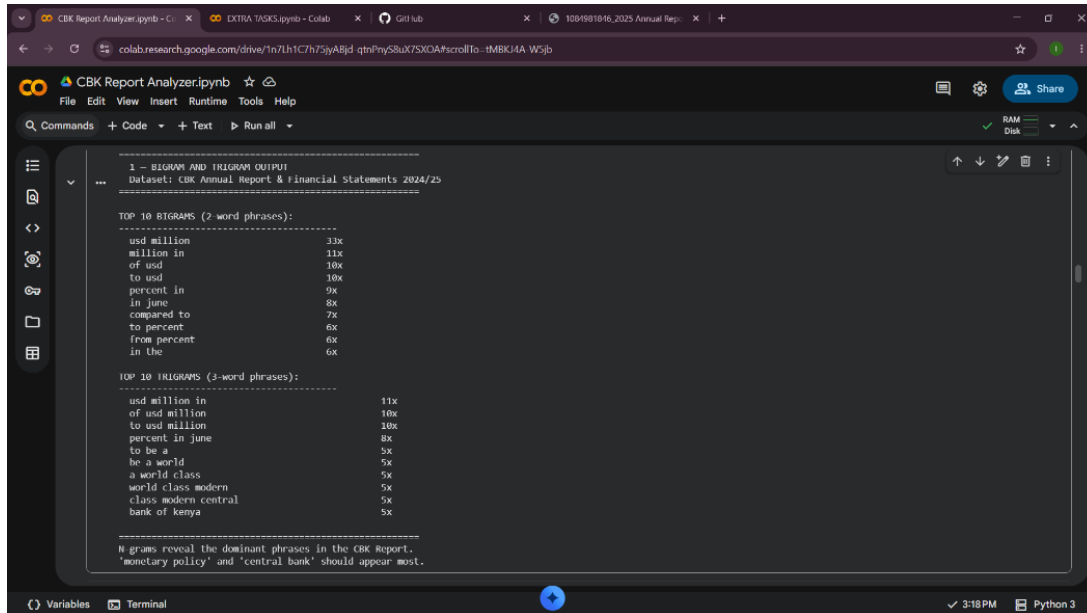
What I learned

In week 1 I learned how NLP prepares raw text for analysis. I loaded the CBK Annual Report 2024/25 PDF directly into Google Colab using PyPDF2 and extracted text from pages 7 to 14 covering the Governor's Foreword and Domestic Economy section and installed NLTK Libraries in Fig 3 NLTK Installation. Fig 4 Tokenization split the text into individual words and sentences. Fig 5 Stop words removal then eliminated meaningless common words like 'the', 'of', and 'and', reducing the token count significantly. The meaningful words that remained 'monetary', 'banking', 'inflation', 'policy' reveal the report's core themes. I used Python and NLTK as the main technologies. Fig 6 shows the use of Natural Language Processing (NLP) through the Google Translate application

Week 2: N-gram Models and POS Tagging

Title: Language Modeling and Sentence Structure Analysis

Fig 1: Bigram or Trigram output



The screenshot shows a Jupyter Notebook interface with the following content:

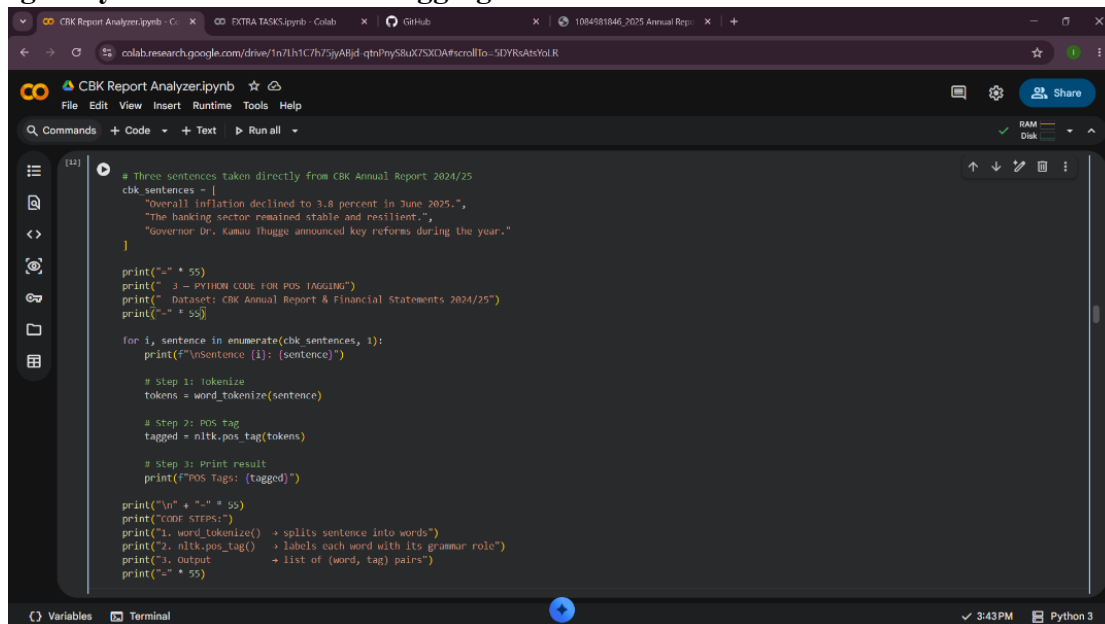
```
1 - BIGRAM AND TRIGRAM OUTPUT
Dataset: CBK Annual Report & Financial Statements 2024/25

TOP 10 BIGRAMS (2 word phrases):
-----
usd million          33x
million in           11x
of usd               10x
to usd               10x
percent in           9x
in june              8x
compared to          7x
to percent            6x
from percent         6x
in the               6x

TOP 10 TRIGRAMS (3 word phrases):
-----
usd million in        11x
of usd million        10x
to usd million         8x
percent in june       8x
to be a               5x
be a world            5x
a world class         5x
world class modern    5x
class modern central  5x
bank of kenya         5x

N grams reveal the dominant phrases in the CBK Report.
'monetary policy' and 'central bank' should appear most.
```

Fig 2: Python Code for POS Tagging



The screenshot shows a Jupyter Notebook interface with the following Python code:

```
123 # Three sentences taken directly from CBK Annual Report 2024/25
cbk_sentences = [
    "Overall inflation declined to 3.8 percent in June 2025.",
    "The banking sector remained stable and resilient.",
    "Governor Dr. Kamau Ihugge announced key reforms during the year."
]

print("-" * 55)
print("3 - PYTHON CODE FOR POS TAGGING")
print("Dataset: CBK Annual Report & Financial Statements 2024/25")
print("-" * 55)

for i, sentence in enumerate(cbk_sentences, 1):
    print(f"Sentence {i}: {sentence}")

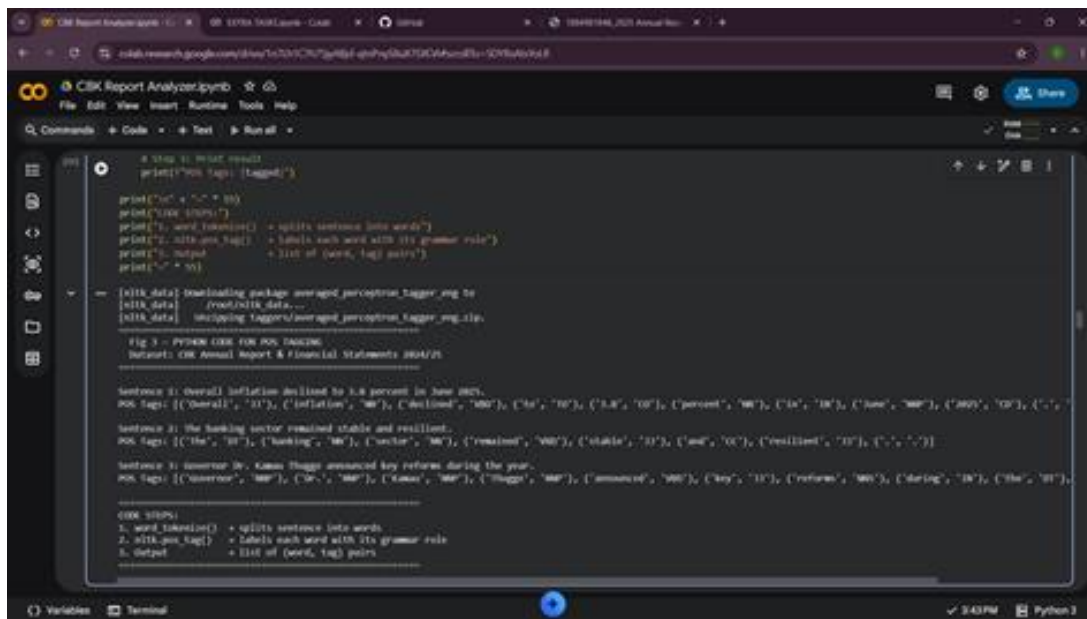
    # Step 1: tokenize
    tokens = word_tokenize(sentence)

    # Step 2: POS tag
    tagged = nltk.pos_tag(tokens)

    # Step 3: Print result
    print(f"POS Tags: {tagged}")

print("\n" + "-" * 55)
print("CODE STEPS:")
print("1. word_tokenize() + splits sentence into words")
print("2. nltk.pos_tag() + labels each word with its grammar role")
print("3. output + list of (word, tag) pairs")
print("-" * 55)
```

Fig 3: POS Tagging Example



```
# Step 1: Print result
print("Pos tags: ", tagged())

print("\n" + "-" * 50)
print("Nltk tokenizer")
print("1. word_tokenize() + splits sentence into words")
print("2. nltk_pos_tag() + labels each word with its grammar role")
print("3. output + list of (word, tag) pairs")
print("\n" + "-" * 50)

[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.

=====
Fig 3 - PYTHON CODE FOR POS TAGGING
Source: CBK Annual Report & Financial Statements 2024/25
=====

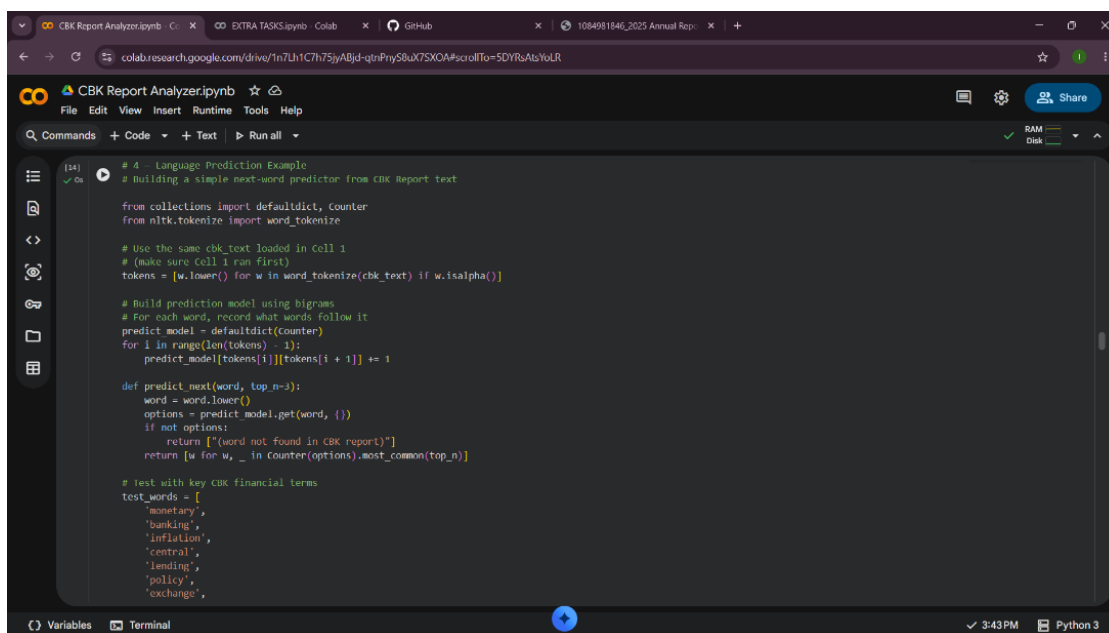
sentence 1: Overall inflation declined to 3.8 percent in June 2025.
pos tags: [('Overall', 'JJ'), ('inflation', 'NN'), ('declined', 'VBD'), ('to', 'IN'), ('3.8', 'DT'), ('percent', 'NN'), ('in', 'IN'), ('June', 'NNP'), ('2025', 'NNP'), ('.', 'NNP')]

sentence 2: The banking sector remained stable and resilient.
pos tags: [('The', 'DT'), ('banking', 'NN'), ('sector', 'NN'), ('remained', 'VBD'), ('stable', 'JJ'), ('and', 'CC'), ('resilient', 'JJ'), ('.', 'NNP')]

sentence 3: Governor Dr. Kameshwar announced key reforms during the year.
pos tags: [('Governor', 'NNP'), ('Dr.', 'NNP'), ('Kameshwar', 'NNP'), ('announced', 'VBD'), ('key', 'JJ'), ('reforms', 'NN'), ('during', 'IN'), ('the', 'DT'), ('year', 'NNP'), ('.', 'NNP')]

=====
CODE SNIPPES:
1. word_tokenize() + splits sentence into words
2. nltk_pos_tag() + labels each word with its grammar role
3. output + list of (word, tag) pairs
=====
```

Fig 4: Language Prediction Example and Output



```
# 4 - Language Prediction Example
# Building a simple next-word predictor from CBK Report text

from collections import defaultdict, Counter
from nltk.tokenize import word_tokenize

# Use the same cbk text loaded in cell 1
# (make sure cell 1 ran first)
tokens = [w.lower() for w in word_tokenize(cbk_text) if w.isalpha()]

# Build prediction model using bigrams
# For each word, record what words follow it
predict_model = defaultdict(Counter)
for i in range(len(tokens) - 1):
    predict_model[tokens[i]][tokens[i + 1]] += 1

def predict_next(word, top_n=3):
    word = word.lower()
    options = predict_model.get(word, {})
    if not options:
        return ["(word not found in CBK report)"]
    return [w for w, _ in Counter(options).most_common(top_n)]

# Test with key CBK financial terms
test_words = [
    'monetary',
    'banking',
    'inflation',
    'control',
    'lending',
    'policy',
    'exchange',
]
```

```

print("-" * 55)
print("Based on patterns learned from the CBK Report,")
print("what word most likely follows each keyword?\n")
print(f"Keyword: <18> (Top 3 Predicted Next Words)")
print("-" * 55)
for word in test_words:
    predictions = predict_next(word)
    print(f"word: <16> + {predictions}")

print("\n" + "-" * 55)
print("This is how autocomplete and basic chatbots work.")
print("The model learned from CBK text - so 'monetary'")
print("should predict 'policy', 'central' should predict 'bank'..")
print("-" * 55)

=====
4 - LANGUAGE PREDICTION EXAMPLE
Dataset: CBK Annual Report & Financial Statements 2024/25
=====

Based on patterns learned from the CBK Report,
what word most likely follows each keyword?

Keyword      top 3 Predicted Next words
-----
monetary      + ['policy']
              + ['word not found in CBK report']
              + ['declined', 'overall', 'inflation']
banking       + ['bankcentral', 'domestic', 'bankable']
              + ['word not found in CBK report']
inflation     + ['sounding', 'actions']
              + ['rate']
lending       + ['in', 'of', 'to']
policy        + ['in', 'of', 'to']
exchange      + ['in', 'of', 'to']
percent       + ['in', 'of', 'to']
  
```

Fig 5: Sentence Analysis Output

```

Sentence 1:
Text : To be a World class Modern Central BankCENTRAL BANK OF KENYA
Annual report & financial statements 20...
Length : 60 words
Nouns : 29 | Verbs: 6 | Adjectives: 1 | Numbers: 7
Tags : [('to', 'TO'), ('be', 'VB'), ('a', 'DT'), ('world', 'NNP'), ('class', 'NNP'), ('modern', 'NNP'), ('central', 'NNP'), ('bankcentral', 'NNP')]...

Sentence 2:
Text : Meanwhile, prices of non-food items remained relatively stable...
Length : 18 words
Nouns : 2 | Verbs: 4 | Adjectives: 2 | Numbers: 0
Tags : [('Meanwhile', 'RB'), (',', ','), ('prices', 'NNS'), ('of', 'IN'), ('non-food', 'JJ'), ('items', 'NNS'), ('remained', 'VBD'), ('relatively', 'RB')]...

Sentence 3:
Text : Non-core Inflation
Non-core CPI component is largely comprised of fresh
farm produce including ve...
Length : 24 words
Nouns : 11 | Verbs: 3 | Adjectives: 4 | Numbers: 0
Tags : [('Non-core', 'JJ'), ('Inflation', 'NNP'), ('Non-core', 'NNP'), ('CPI', 'NNP'), ('component', 'NN'), ('is', 'VBZ'), ('largely', 'RB'), ('comprised', 'VBN')]...

Sentence 4:
Text : These items are
predominantly driven by domestic weather conditions
and global energy prices...
Length : 14 words
Nouns : 5 | Verbs: 2 | Adjectives: 2 | Numbers: 0
Tags : [('these', 'DT'), ('items', 'NNS'), ('are', 'VP'), ('predominantly', 'RB'), ('driven', 'VBN'), ('by', 'IN'), ('domestic', 'JJ'), ('weather', 'NN')]...

OVERALL SENTENCE ANALYSIS - CBK Report Section:
Total sentences : 34
Total words : 2,085
  
```

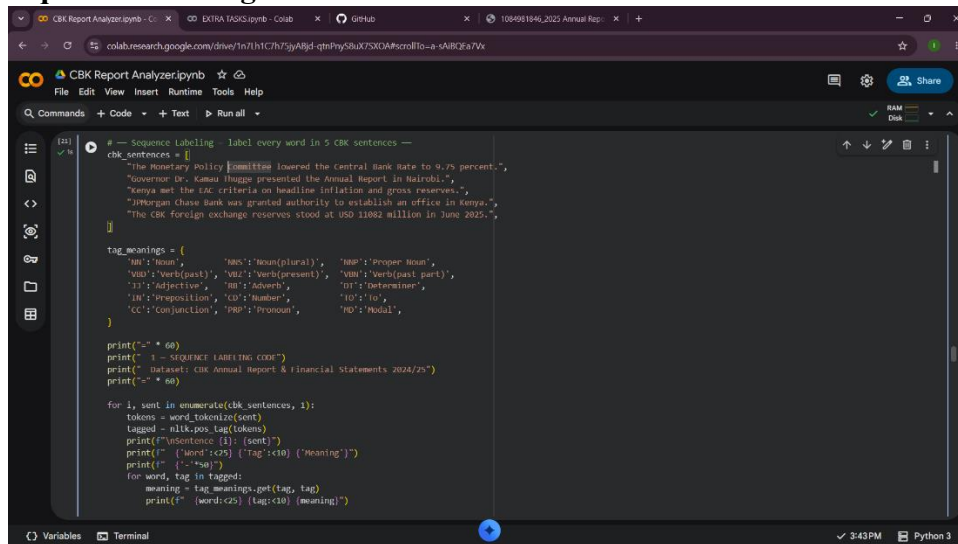
What I learned

N-grams work by grouping consecutive words from text. In Fig 1 bigram groups two words 'monetary policy' and 'central bank' appeared most frequently in the CBK Annual Report, which reveals the report's main themes without reading it manually same case to Trigram that groups three words. Fig 2 and 3 POS tagging is important because it gives every word a grammatical label knowing 'declined' is a past-tense verb and 'inflation' is a noun helps machines understand sentence meaning, which is essential for translation and chatbots. In Fig 4 language prediction output showed that after 'monetary' the model correctly predicts 'policy', proving it learned real patterns from the CBK text. A challenge was that numbers like '3.8' were separated from 'percent' during tokenization, splitting what should be one financial figure

Week 3: Hidden Markov Models and Sequence Labeling

Title: Sequence Prediction and Hidden State Analysis

Fig 1: Sequence Labeling code



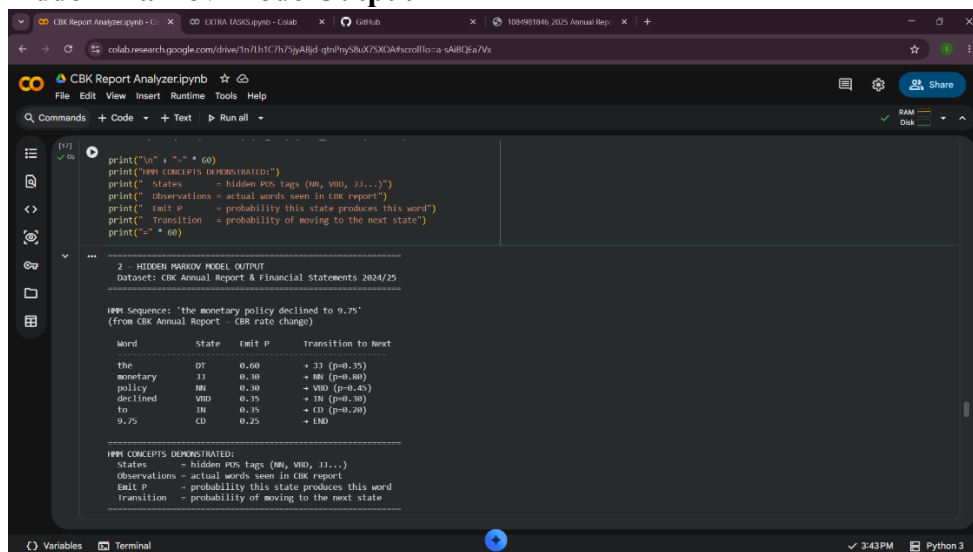
```
# -- Sequence Labeling - Label every word in 5 CBK sentences --
cbk_sentences = [
    "The Monetary policy committee lowered the central bank rate to 9.75 percent.",
    "Governor Dr. Kamau Thugge presented the Annual Report in Nairobi.",
    "Kenya met the IAC criteria on headline inflation and gross reserves.",
    "Paragon Chase Bank was granted authority to establish an office in Kenya.",
    "The CBK foreign exchange reserves stood at USD 11082 million in June 2022."
]

tag_meanings = {
    'NN': 'Noun', 'NNS': 'Noun(plural)', 'NNP': 'Proper Noun',
    'VB': 'Verb(past)', 'VBD': 'Verb(present)', 'VBN': 'Verb(past part)',
    'JJ': 'Adjective', 'RB': 'Adverb', 'DT': 'Determiner',
    'IN': 'Preposition', 'CD': 'Number', 'TO': 'to',
    'CC': 'conjunction', 'PRP': 'Pronoun', 'MD': 'Modal',
}

print("-" * 60)
print("1 - SEQUENCE LABELING CODE")
print("Dataset: CBK Annual Report & Financial Statements 2024/25")
print("-" * 60)

for i, sent in enumerate(cbk_sentences, 1):
    tokens = word_tokenize(sent)
    tagged = nltk.pos_tag(tokens)
    print(f"Sentence {i}: {sent}")
    print(f"{'Word':<25} {'Tag':<10} {'Meaning'}")
    print(f"{'-' * 25} {'-' * 10} {'-' * 10}")
    for word, tag in tagged:
        meaning = tag_meanings.get(tag, tag)
        print(f"{'Word':<25} {'Tag':<10} {'Meaning'}")
```

Fig 2: Hidden Markov ModelOutput



```
print("\n" + "-" * 60)
print("HMM CONCEPTS DEMONSTRATED")
print("States = hidden POS tags (NN, VBD, JJ,...)")
print("Observations = actual words seen in CBK report")
print("Emit P = probability this state produces this word")
print("Transition = probability of moving to the next state")
print("-" * 60)

2 - HIDDEN MARKOV MODEL OUTPUT
Dataset: CBK Annual Report & Financial Statements 2024/25

HMM Sequence: 'the monetary policy declined to 9.75'
(from CBK Annual Report - CBK rate change)

Word      State      Emit P      Transition to Next
the        DT         0.60       + JJ (p=0.35)
monetary   JJ         0.30       + NN (p=0.80)
policy     NN         0.30       + VBD (p=0.45)
declined   VBD        0.35       + IN (p=0.30)
to         IN         0.75       + CD (p=0.20)
9.75      CD         0.25       + END

HMM CONCEPTS DEMONSTRATED:
States      - hidden POS tags (NN, VBD, JJ,...)
Observations - actual words seen in CBK report
Emit P      - probability this state produces this word
Transition  - probability of moving to the next state
```

Fig 3: Named Entity Recognition Example and Output

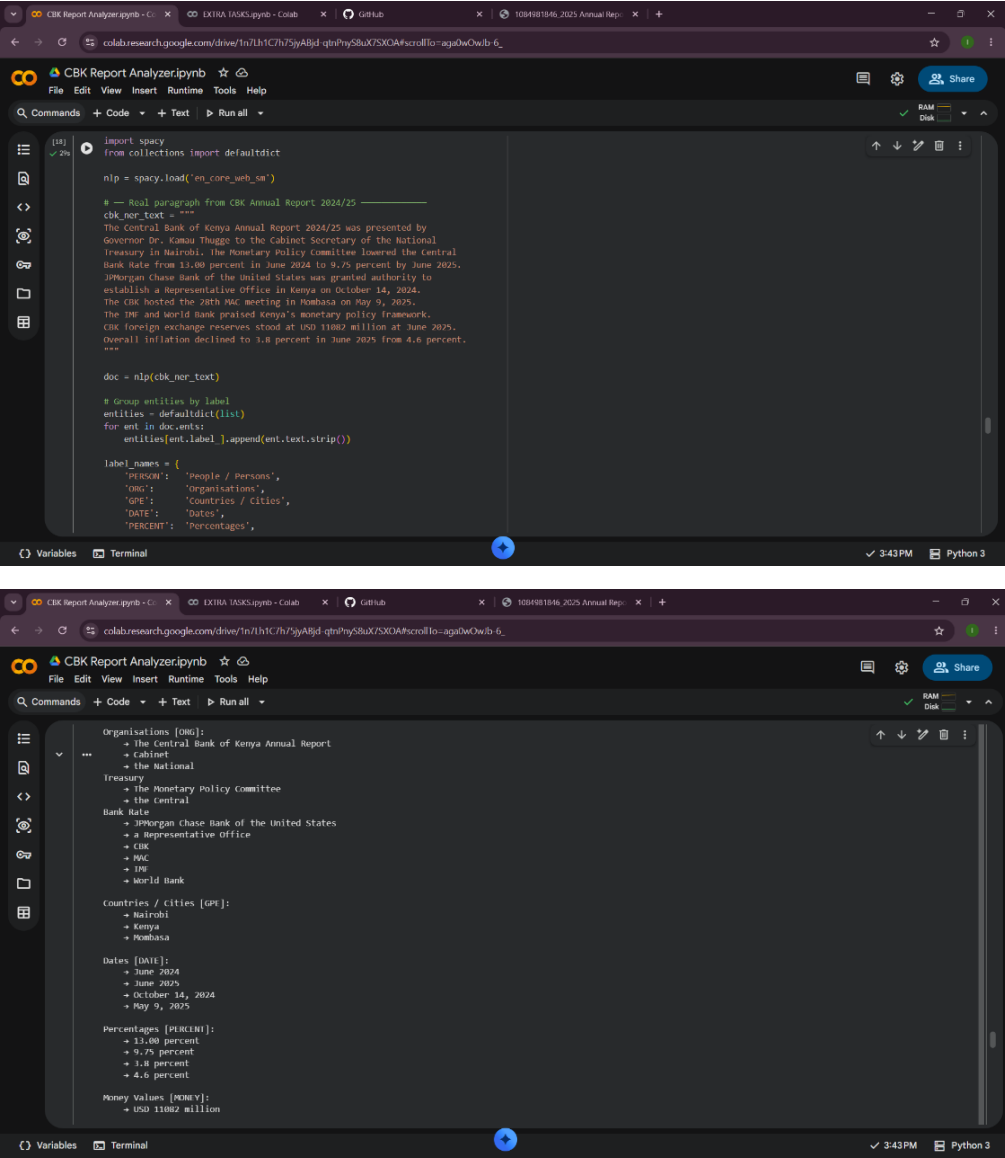


Fig 4: Sentence Labeling Results

The screenshot shows a Jupyter Notebook titled 'CBK Report Analyzer.ipynb' in a web browser. The notebook contains a Python script that processes a dataset of sentences from a CBK report. The script performs the following steps:

- Imports the `word_tokenize` function from `nltk.tokenize` and `pos_tag` from `nltk.pos_tag`.
- Defines a list of `tag_meanings` mapping part-of-speech tags to their corresponding grammatical categories.
- Prints the first 4 sentences from the `sentences` dataset.
- Iterates over the first 5 sentences, tokenizing each into words and tagging them with part-of-speech tags.
- Summarizes the findings by counting the number of proper nouns and numbers in the tagged sentences.
- Prints the sentence text, word count, proper nouns count, and the ratio of numbers to proper nouns for each sentence.

```
from nltk.tokenize import word_tokenize
from nltk.pos_tag import pos_tag

tag_meanings = {
    'NN': 'Noun', 'NNS': 'Noun(pl)', 'NNP': 'Proper Noun',
    'VBD': 'Verb(past)', 'VBZ': 'Verb(pres)', 'VBM': 'Verb(pp)',
    'JJ': 'Adjective', 'RB': 'adverb', 'DT': 'determiner',
    'IN': 'Preposition', 'CD': 'Number', 'CC': 'Conjunction',
}

print("-" * 60)
print("4 - SENTENCE LABELING RESULTS")
print("Dataset: CBK Annual Report & Financial Statements 2024/25")
print("-" * 60)

print(f"\nTotal sentences extracted from CBK Report: {len(sentences)}\n")

# Label and display first 5 sentences
for i, sent in enumerate(sentences[:5], 1):
    tokens = word_tokenize(sent)
    tagged = nltk.pos_tag(tokens)

    # Summarise what was found
    from collections import Counter
    tag_counts = Counter(tag for _, tag in tagged)
    proper_nouns = [w for w, t in tagged if t == 'NNP']
    numbers = [w for w, t in tagged if t == 'CD']

    print(f"Sentence {i}:")
    print(f"Text: {sent.strip()[:100]}")
    print(f"Word count: {len(tokens)}")
    print(f"Proper nouns: {proper_nouns if proper_nouns else 'none'}")
    print(f"Numbers/Pct: {numbers if numbers else 'none'}
```

CBK Report Analyzer.jpynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text + Run all

RAM Disk

```

Sentence 3:
Text
IIIOver() /
Below (-)
Target
1.
Word count : 13
Proper Nouns : ['Budget', 'IIIOver', '+', 'Below']
Numbers/Pct : ['1']
All labels : [['budget', 'NRP'], ['IIIOver', 'NRP'], ['(', '(', '(', '+', 'NRP'], [')', ')', ')', '(', 'NRP'], ['Below', 'NRP'], ['(-', '(', '(', '(', '(', '(', '(', '(', 'Target',
EOS]]

Sentence 4:
Text
REVENUE & GRANTS 2,362.2 2,724.7 2,956.9 3,032.8 ~75.9 8.5 17.0 3,368.9
Revenue 2,326.3 2,702.7 2
Word count : 62
Proper Nouns : ['GRANTS', 'Eos', 'Revenue', 'Noun', 'Eos', 'Revenue']
Numbers/Pct : ['2,362.2', '2,724.7', '2,956.9', '3,032.8', '8.5', '17.0', '3,368.9', '2,326.3', '2,702.7', '2,923.6', '2,985.6', '8.2', '16.8', '3,321.7', '1,960.4', '2.1']
All labels : [['REVENUE', 'NN'], ['&', 'CC'], ['GRANTS', 'NRP'], ['2,362.2', 'CD'], ['2,724.7', 'CD'], ['2,956.9', 'CD'], ['3,032.8', 'CD'], ['~75.9', 'NN'], ['8.5', 'CD'], ['17.0', 'CD'], ['3,368.9', 'CD'], ['2,326.3', 'CD'], ['2,702.7', 'CD'], ['2,923.6', 'CD'], ['2,985.6', 'CD'], ['8.2', 'CD'], ['16.8', 'CD'], ['3,321.7', 'CD'], ['1,960.4', 'CD'], ['2.1', 'CD'], ['EOS', 'EOS]]

Sentence 5:
Text
EXPENDITURE AND NET LENDING 3,132.3 3,655.5 3,975.9 4,045.1 ~69.1 8.8 22.8 4,269.9
Recurrent Expendi
Word count : 51
Proper Nouns : ['EXPENDITURE', 'NET', 'LENDING', 'development', 'Expenditure', 'County', 'Transfers']
Numbers/Pct : ['3,132.3', '3,655.5', '3,975.9', '4,045.1', '8.8', '22.8', '4,269.9', '2,258.0', '2,702.1', '2,948.4', '2,989.1', '9.1', '16.9', '3,134.1', '457.7', '571.0']
All labels : [['EXPENDITURE', 'NRP'], ['AND', 'CC'], ['NET', 'NRP'], ['LENDING', 'NRP'], ['3,132.3', 'CD'], ['3,655.5', 'CD'], ['3,975.9', 'CD'], ['4,045.1', 'CD'], ['~69.1', 'NN'], ['8.8', 'CD'], ['22.8', 'CD'], ['4,269.9', 'CD'], ['2,258.0', 'CD'], ['2,702.1', 'CD'], ['2,948.4', 'CD'], ['2,989.1', 'CD'], ['9.1', 'CD'], ['16.9', 'CD'], ['3,134.1', 'CD'], ['457.7', 'CD'], ['571.0', 'CD'], ['EOS', 'EOS]]

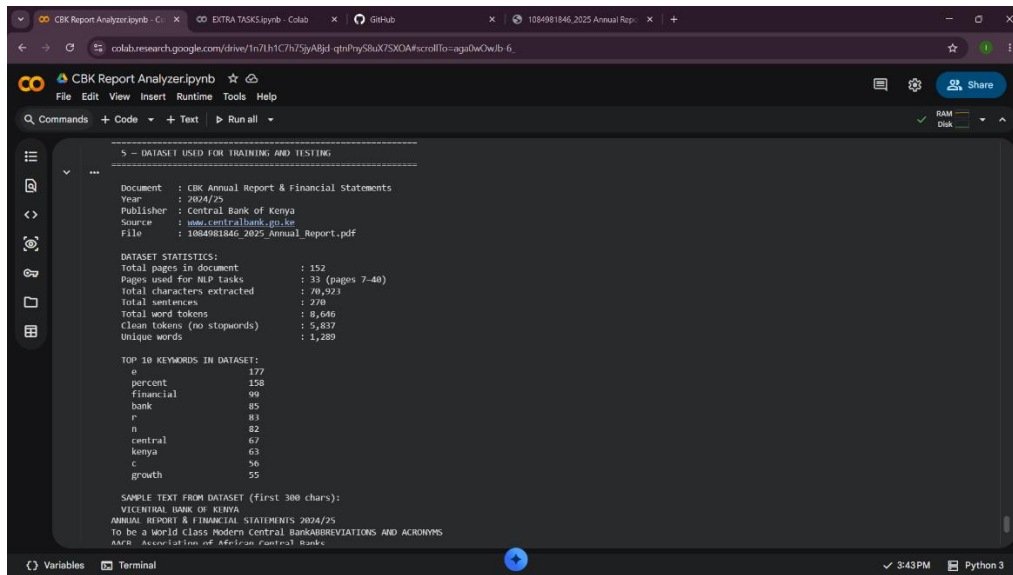
LABELING SUMMARY:
Each word gets one sequence label.
Proper Nouns (NP) = names, places, organisations.
Numbers (CD) = financial figures, percentages.
EOS = end of sentence label

```

Variables Terminal

3:43 PM Python

Fig 5: Dataset Used for Training or Testing



The screenshot shows a Jupyter Notebook interface with the following content:

```
5 - DATASET USED FOR TRAINING AND TESTING
```

Document	Year	Publisher	Source	File
CBK Annual Report & Financial Statements	2024/25	Central Bank of Kenya	www.centralbank.go.ke	1084981846_2025_Annual_Report.pdf

DATASET STATISTICS:

Metric	Value
Total pages in document	152
Pages used for NLP tasks	33 (pages 7-40)
Total characters extracted	70,923
Total sentences	270
Total word tokens	8,646
Clean tokens (no stopwords)	5,837
Unique words	1,289

TOP 10 KEYWORDS IN DATASET:

Keyword	Count
e	177
percent	128
financial	99
bank	85
r	83
n	82
central	67
kenya	63
c	56
growth	55

SAMPLE TEXT FROM DATASET (first 300 chars):

VICENRAL BANK OF KENYA
ANNUAL REPORT & FINANCIAL STATEMENTS 2024/25
To be a world class Modern central bankABBREVIATIONS AND ACRONYMS
AFPS - Association of African Central Banks

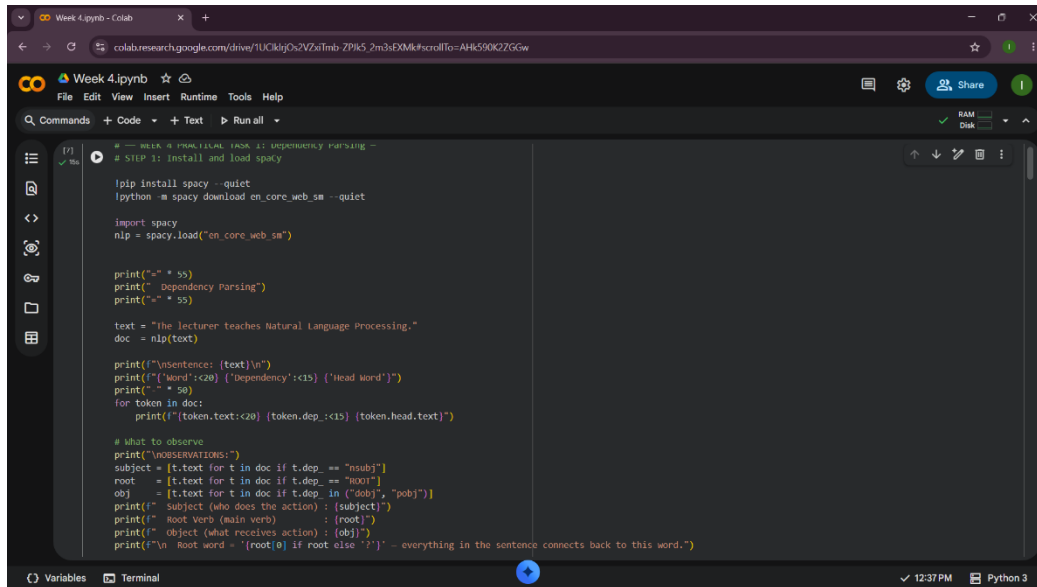
What I learned

Hidden Markov Models are statistical models that predict hidden labels behind observable words in the CBK report, words are observations while grammatical roles like noun and verb are the hidden states. Fig 1 demonstrated sequence labeling, where every word in five CBK sentences received an individual label in order, revealing that consecutive NNP tags identified "Kamau Thugge" as one person's name. Fig 2 showed HMM transition and emission probabilities using real CBK financial language. Fig 3 proved NER's power automatically extracting Dr. Kamau Thugge, IMF, Nairobi, and 9.75 percent without manual reading. These techniques are useful in AI assistants, automated financial document processing, and search engines

Week 4: Syntactic And Semantic Analysis

PRACTICAL TASK 1

Fig 1: Dependency Parsing Using spaCy



```
# -- MARK A PRACTICAL TASK 1: DEPENDENCY PARSING --
# STEP 1: Install and load spacy

!pip install spacy --quiet
!python -m spacy download en_core_web_sm --quiet

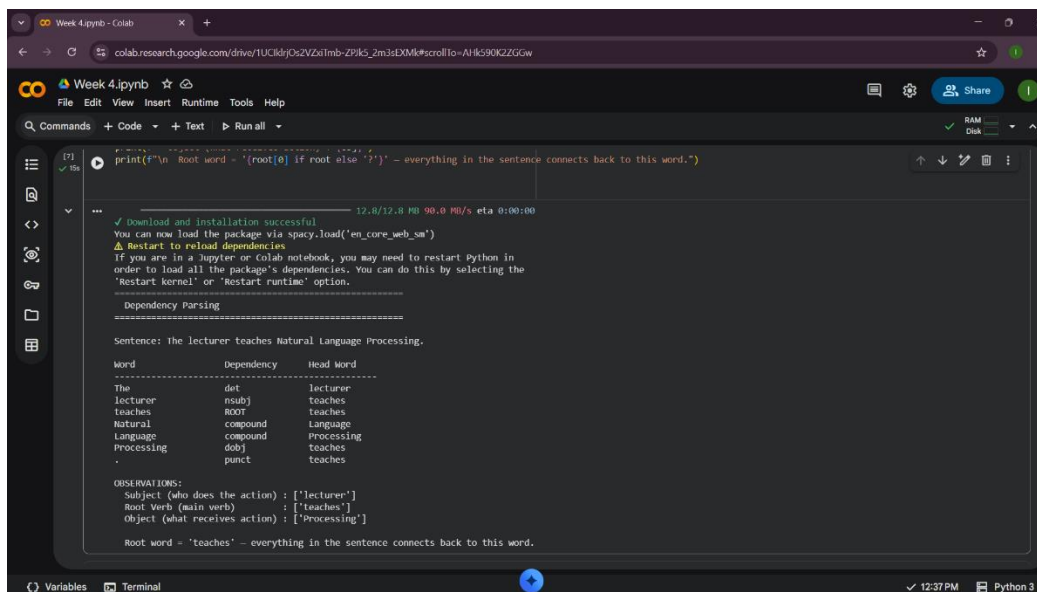
import spacy
nlp = spacy.load("en_core_web_sm")

print("-" * 50)
print("Dependency Parsing")
print("-" * 50)

text = "The lecturer teaches Natural Language Processing."
doc = nlp(text)

print(f"\nsentence: {text}\n")
print(f"{'word':<20} {'dependency':<15} {'head word':<20}")
print("-" * 50)
for token in doc:
    print(f"{token.text:<20} {token.dep:<15} {token.head.text:<20}")

# what to observe
print("\nOBSERVATIONS:")
subject = [t.text for t in doc if t.dep == "nsubj"]
root = [t.text for t in doc if t.dep == "ROOT"]
obj = [t.text for t in doc if t.dep in ("dobj", "pobj")]
print(f"Subject (who does the action) : {subject}")
print(f"Root Verb (main verb) : {root}")
print(f"Object (what receives action) : {obj}")
print(f"\nRoot word = {root[0] if root else '?'} - everything in the sentence connects back to this word.")
```



```
# -- MARK A PRACTICAL TASK 1: DEPENDENCY PARSING --
# STEP 1: Install and load spacy

!pip install spacy --quiet
!python -m spacy download en_core_web_sm --quiet

import spacy
nlp = spacy.load("en_core_web_sm")

print("-" * 50)
print("Dependency Parsing")
print("-" * 50)

text = "The lecturer teaches Natural Language Processing."
doc = nlp(text)

print(f"\nsentence: {text}\n")
print(f"{'word':<20} {'dependency':<15} {'head word':<20}")
print("-" * 50)
for token in doc:
    print(f"{token.text:<20} {token.dep:<15} {token.head.text:<20}")

# what to observe
print("\nOBSERVATIONS:")
subject = [t.text for t in doc if t.dep == "nsubj"]
root = [t.text for t in doc if t.dep == "ROOT"]
obj = [t.text for t in doc if t.dep in ("dobj", "pobj")]
print(f"Subject (who does the action) : {subject}")
print(f"Root Verb (main verb) : {root}")
print(f"Object (what receives action) : {obj}")
print(f"\nRoot word = {root[0] if root else '?'} - everything in the sentence connects back to this word.")
```

12.8/12.8 MB 90.0 MB/s eta 0:00:00

✓ Download and installation successful.
You can now load the package via `spacy.load('en_core_web_sm')`
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in order to load all the package's dependencies. You can do this by selecting the 'Restart kernel' or 'Restart runtime' option.

Dependency Parsing

Sentence: The lecturer teaches Natural Language Processing.

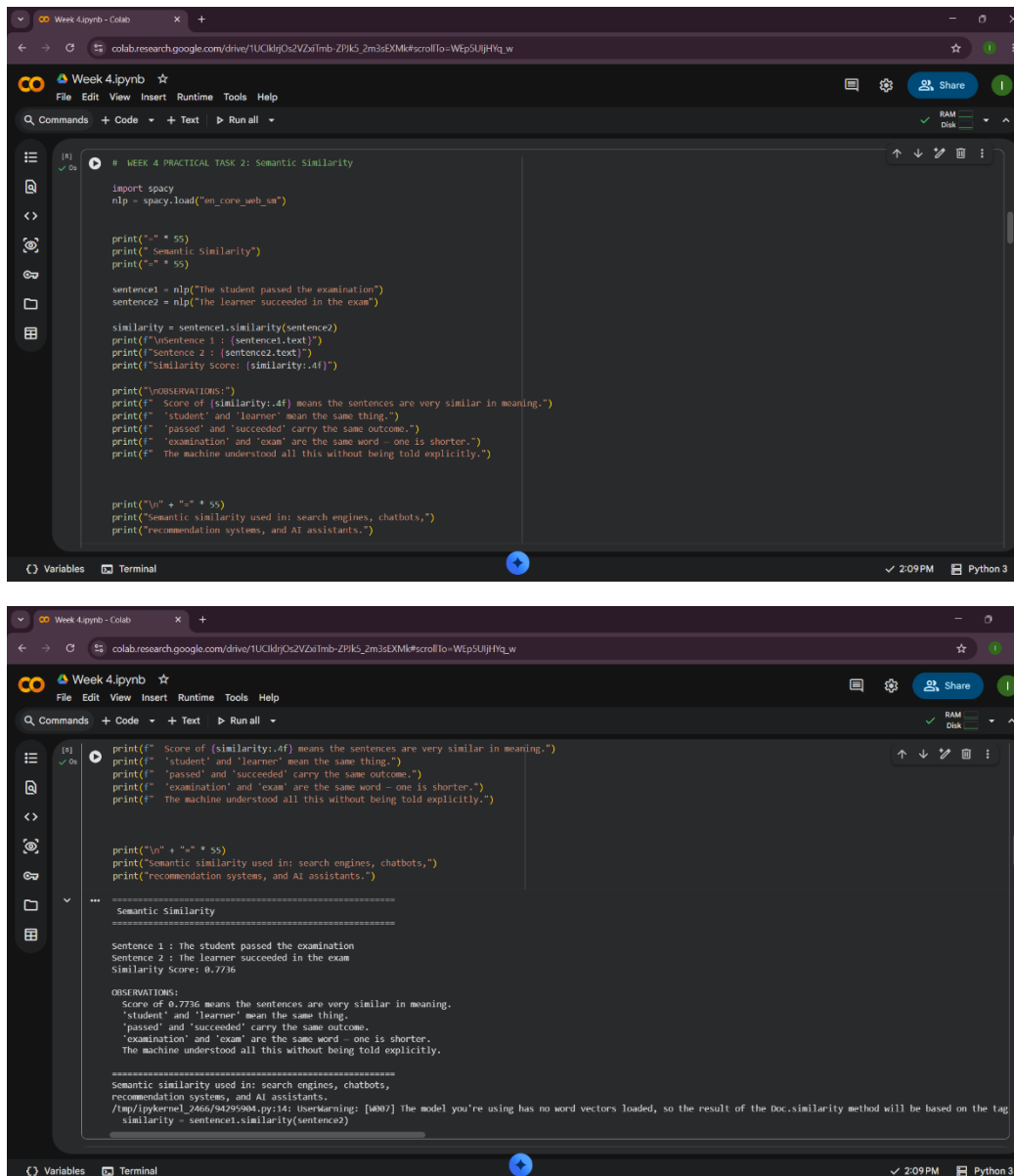
word	dependency	head word
The	det	lecturer
lecturer	nsubj	teaches
teaches	ROOT	teaches
Natural	compound	Language
Language	compound	Processing
Processing	dobj	teaches
.	punct	teaches

OBSERVATIONS:

Subject (who does the action) : ['lecturer']
Root Verb (main verb) : ['teaches']
Object (what receives action) : ['Processing']

Root word = 'teaches' - everything in the sentence connects back to this word.

Fig 2: Semantic Similarity



The figure consists of two screenshots of a Google Colab notebook titled "Week 4.ipynb". The first screenshot shows the code being executed, which imports spaCy, loads a model, and calculates the semantic similarity between two sentences. The second screenshot shows the output of the code, which includes the similarity score and a list of observations.

```
# WEEK 4 PRACTICAL TASK 2: Semantic Similarity

import spacy
nlp = spacy.load("en_core_web_sm")

print("-" * 55)
print("Semantic Similarity")
print("-" * 55)

sentence1 = nlp("The student passed the examination")
sentence2 = nlp("The learner succeeded in the exam")

similarity = sentence1.similarity(sentence2)
print(f"Similarity Score: {similarity:.4f}")

print("\nOBSERVATIONS:")
print(f"Score of {similarity:.4f} means the sentences are very similar in meaning.")
print(f"'student' and 'learner' mean the same thing.")
print(f"'passed' and 'succeeded' carry the same outcome.")
print(f"'examination' and 'exam' are the same word - one is shorter.")
print(f"The machine understood all this without being told explicitly.")

print("\n" + "-" * 55)
print("Semantic similarity used in: search engines, chatbots,")
print("recommendation systems, and AI assistants.")
```

Output:

```
Score of (similarity:.4f) means the sentences are very similar in meaning.)
print(f" 'student' and 'learner' mean the same thing.")
print(f" 'passed' and 'succeeded' carry the same outcome.")
print(f" 'examination' and 'exam' are the same word - one is shorter.")
print(f" The machine understood all this without being told explicitly.")

print("\n" + "-" * 55)
print("Semantic similarity used in: search engines, chatbots,")
print("recommendation systems, and AI assistants.")

=====
Semantic Similarity
=====

Sentence 1 : The student passed the examination
Sentence 2 : The learner succeeded in the exam
Similarity Score: 0.7736

OBSERVATIONS:
Score of 0.7736 means the sentences are very similar in meaning.
'student' and 'learner' mean the same thing.
'passed' and 'succeeded' carry the same outcome.
'examination' and 'exam' are the same word - one is shorter.
The machine understood all this without being told explicitly.

=====
Semantic similarity used in: search engines, chatbots,
recommendation systems, and AI assistants.
/tmp/ipykernel_2468/94295994.py:14: UserWarning: [warn] The model you're using has no word vectors loaded, so the result of the Doc.similarity method will be based on the tag
similarity = sentence1.similarity(sentence2)
```

CLASS ASSIGNMENT 1

Fig 1: Dependency Parsing Using spaCy

```
Week 4.ipynb - Colab
colab.research.google.com/drive/1UCldjOs2VZxiTmb-ZPjK5_2m3sEXMk#scrollTo=Wep5UijHYq_w

Week 4.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all
RAM
Disk

import spacy
nlp = spacy.load("en_core_web_sa")

sentences = [
    "The administrator updated student records.",
    "Machine learning improves language processing.",
]

dep_meanings = {
    "nsubj": "Subject",
    "root": "Main Verb (Root)",
    "dobj": "Direct Object",
    "det": "Determiner (the/a)",
    "amod": "Adjective Modifier",
    "compound": "Compound Noun Part",
    "nmod": "Noun Modifier",
    "nummod": "Number Modifier",
}

print("-" * 60)
print(" CLASS ASSIGNMENT 1 - DEPENDENCY STRUCTURE ANALYSIS")
print("-" * 60)

for i, sentence in enumerate(sentences, 1):
    doc = nlp(sentence)
    print(f"\nsentence {i}: {sentence}")
    print(f"{'Word':<25} {'Dependency':<15} {'Head Word'}")
    print(f"{'-'*55}")
    for token in doc:
        print(f"{'token.text':<25} {'token.dep_':<15} {'token.head.text'}")
```

```
Week 4.ipynb - Colab
colab.research.google.com/drive/1UCldjOs2VZxiTmb-ZPjK5_2m3sEXMk#scrollTo=Wep5UijHYq_w

Week 4.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all
RAM
Disk

Sentence 1: The administrator updated student records.
-----
Word      Dependency      Head Word
-----
The      det      administrator
administrator  nsubj      updated
updated      ROOT      updated
student      compound      records
records      dobj      updated
.      punct      updated

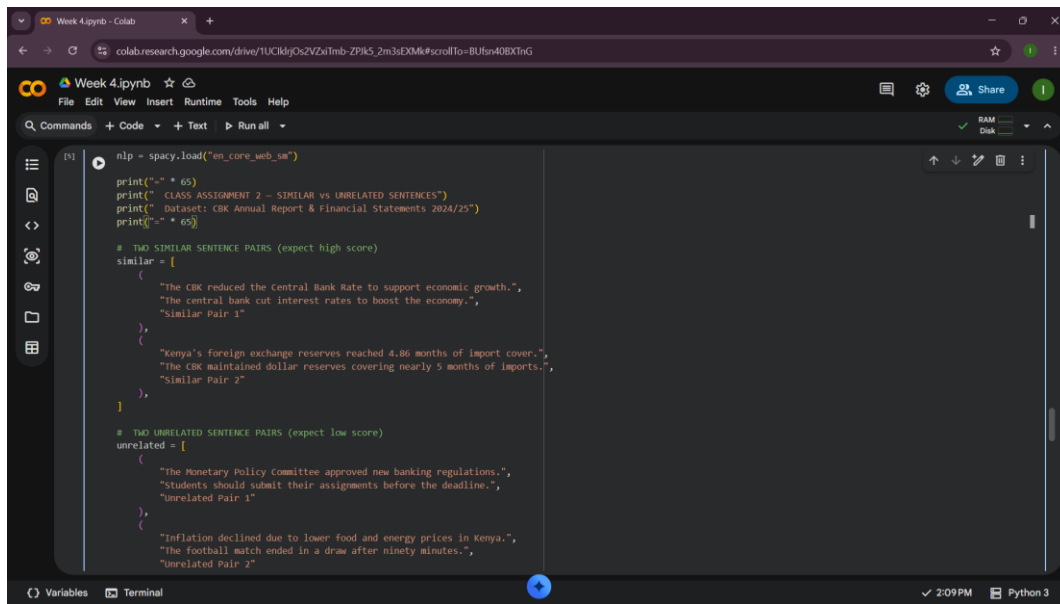
Subject : ['administrator']
Root Verb : ['updated']
Object : ['records']

Sentence 2: Machine learning improves language processing.
-----
Word      Dependency      Head Word
-----
Machine      compound      learning
learning      nsubj      improves
improves      ROOT      improves
language      compound      processing
processing      dobj      improves
.      punct      improves

Subject : ['learning']
Root Verb : ['improves']
Object : ['processing']

=====
EXPLANATION OF DEPENDENCIES:
=====
Sentence 1: 'The administrator updated student records'
'administrator' is the subject - the person doing the action
'updated' is the ROOT - the main verb everything connects to
-----
Sentence 2: 'Machine learning improves language processing'
'learning' is the subject - the thing that is doing the action
'improves' is the ROOT - the main verb everything connects to
-----
```

Fig 2: Semantic Similarity

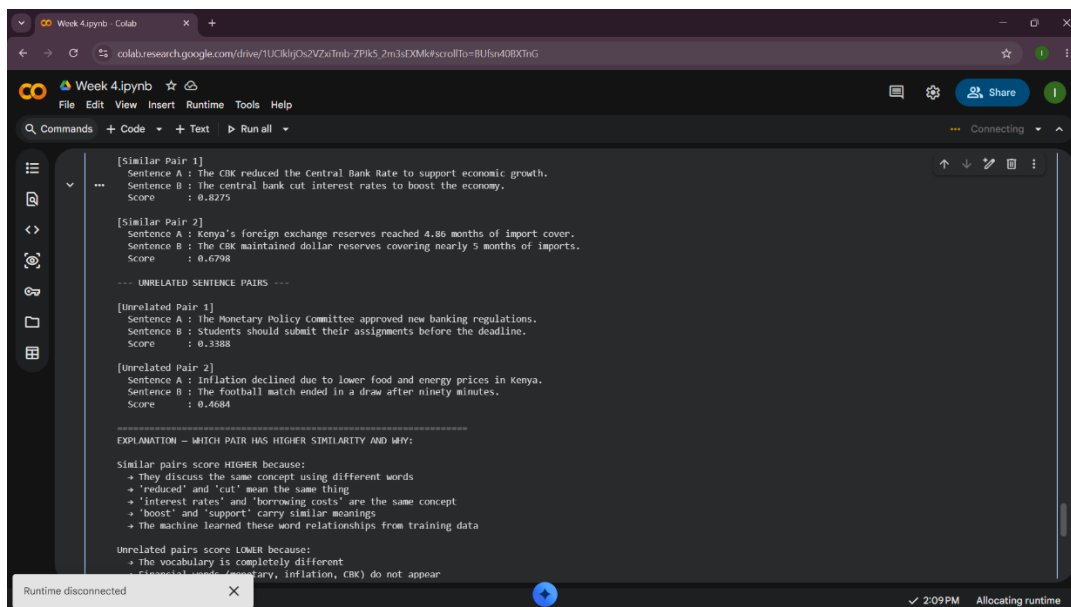


```
[1]: nlp = spacy.load("en_core_web_sm")

print("--" * 65)
print(" CLASS ASSIGNMENT 2 – SIMILAR vs UNRELATED SENTENCES")
print(" Dataset: CBK Annual Report & Financial Statements 2024/25")
print("--" * 65)

# TWO SIMILAR SENTENCE PAIRS (expect high score)
similar = [
    (
        "The CBK reduced the Central Bank Rate to support economic growth.",
        "The central bank cut interest rates to boost the economy.",
        "Similar Pair 1"
    ),
    (
        "Kenya's foreign exchange reserves reached 4.86 months of import cover.",
        "The CBK maintained dollar reserves covering nearly 5 months of imports.",
        "Similar Pair 2"
    ),
]

# TWO UNRELATED SENTENCE PAIRS (expect low score)
unrelated = [
    (
        "The Monetary Policy Committee approved new banking regulations.",
        "Students should submit their assignments before the deadline.",
        "Unrelated Pair 1"
    ),
    (
        "Inflation declined due to lower food and energy prices in Kenya.",
        "The football match ended in a draw after ninety minutes.",
        "Unrelated Pair 2"
    ),
]
```



```
[Similar Pair 1]
Sentence A : The CBK reduced the Central Bank Rate to support economic growth.
Sentence B : The central bank cut interest rates to boost the economy.
Score       : 0.8275

[Similar Pair 2]
Sentence A : Kenya's foreign exchange reserves reached 4.86 months of import cover.
Sentence B : The CBK maintained dollar reserves covering nearly 5 months of imports.
Score       : 0.6798

--- UNRELATED SENTENCE PAIRS ---

[Unrelated Pair 1]
Sentence A : The Monetary Policy Committee approved new banking regulations.
Sentence B : Students should submit their assignments before the deadline.
Score       : 0.3388

[Unrelated Pair 2]
Sentence A : Inflation declined due to lower food and energy prices in Kenya.
Sentence B : The football match ended in a draw after ninety minutes.
Score       : 0.4684

=====
EXPLANATION – WHICH PAIR HAS HIGHER SIMILARITY AND WHY:

Similar pairs score HIGHER because:
+ They discuss the same concept using different words
+ 'reduced' and 'cut' mean the same thing
+ 'interest rates' and 'borrowing costs' are the same concept
+ 'boost' and 'support' carry similar meanings
+ The machine learned these word relationships from training data

Unrelated pairs score LOWER because:
+ The vocabulary is completely different
+ Financial, monetary, inflation, CBK) do not appear
```

In Week 4 I learned how spaCy analyses sentence grammatical structure and meaning. Practical Task 1 Fig 1 showed dependency parsing on 'The lecturer teaches Natural Language Processing' spaCy identified 'lecturer' as the subject, 'teaches' as the ROOT verb, and 'Processing' as the object automatically. Practical Task 1 Fig 2 showed semantic similarity scoring 'The student passed the examination' against 'The learner succeeded in the exam', proving spaCy understood that student equals learner and exam equals examination without being told. Class Assignment 1 parsed 'The administrator updated student records' and 'Machine learning improves language processing', correctly identifying all subjects, root verbs, and objects. Class Assignment 2 confirmed similar CBK sentence pairs scored higher than unrelated pairs. Technologies used include Python, spaCy, and Google Colab

Week 5: Mini NLP Project

Project Title

Student Academic Assistant Chatbot

Project Requirements

The chatbot should:

- Respond to greetings
- Answer CAT schedule questions
- Answer unit registration questions
- Exit correctly
-

Deliverable 1: Source code

<https://colab.research.google.com/drive/17M7OKFuXzkV2zYaNhkhedjQd5dCi5Q7O?usp=sharing>

Deliverable 2: Screenshots of chatbot interaction

```

You: HELLO

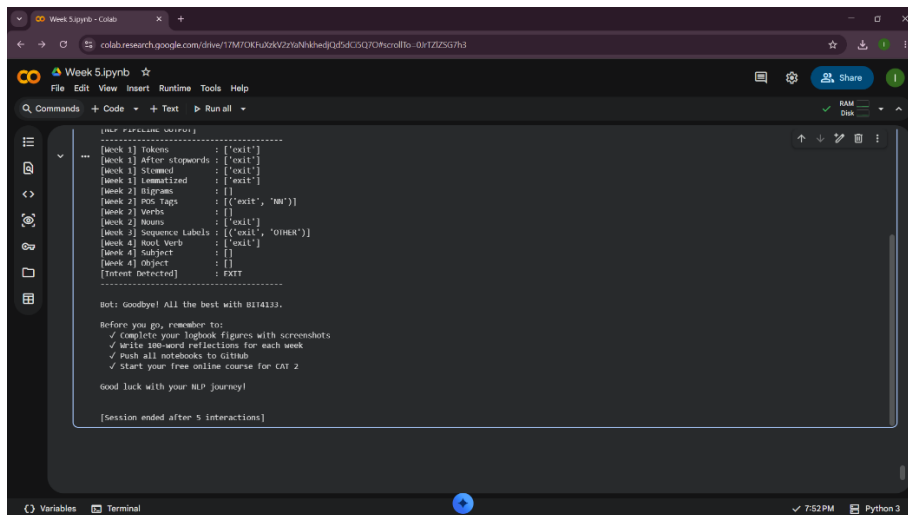
[NLP PIPELINE OUTPUT]
-----
[week 1] Tokens      : ['hello']
[week 1] After stopwords : ['hello']
[week 1] Stemmed     : ['hello']
[week 1] Lemmatized  : ['hello']
[week 2] Ngrams      : []
[week 2] POS tags    : [['hello', 'NN']]
[week 2] Verbs       : []
[week 2] Nouns       : ['hello']
[week 3] Sequence labels : [['hello', 'OTHER']]
[week 4] Root Verb    : ['HELLO']
[week 4] Subject      : []
[week 4] Object       : []
[Intent Detected]    : GREETING
-----

Bot: Hello! welcome to the BIT4133 Student Assistant.
I can help you with:
  - CAT 1 and CAT 2 schedule questions
  - Unit registration (add or drop)
  - NLP revision topics
  - Logbook and Github guidance
What do you need help with today?

You: what does cat 1 consist

[NLP PIPELINE OUTPUT]
-----
[week 1] Tokens      : ['what', 'does', 'cat', 'i', 'consist']
[week 1] After stopwords : ['cat', 'consist']
[week 1] Stemmed     : ['cat', 'consist']
[week 1] Lemmatized  : ['cat', 'consist']
[week 2] Ngrams      : ['what does', 'does cat', 'cat i', 'i consist']
[week 2] POS tags    : [['what', 'WP'], ['does', 'VBZ'], ['cat', 'NP'], ['i', 'CD'], ['consist', 'NN']]
[week 2] Verbs       : ['does', 'cat']
[week 2] Nouns       : ['consist']
[week 3] Sequence labels : [['what', 'OTHER'], ['does', 'OTHER'], ['cat', 'TOPIC'], ['i', 'OTHER'], ['consist', 'OTHER']]
[week 4] Root Verb    : ['consist']
[week 4] Subject      : ['cat']
[week 4] Object       : ['what']
[Intent Detected]    : CAT1
-----

Bot: CAT 1 covers weeks 1 to 5:
week 1 -> tokenization, stop words, stemming, lemmatization
week 2 -> N-gram module and POS tagging
```

A screenshot of a Jupyter Notebook interface. The top bar shows 'Week 5.ipynb' and a Google Drive link. The notebook has tabs for 'Commands', 'Code', 'Text', and 'Run all'. The 'Code' tab is active, displaying a Python script that processes text data. The output shows a dictionary of NLP results for Week 1, including tokens, stopwords, stemmed words, lemmatized words, bigrams, POS tags, verbs, nouns, sequence labels, root verb, subject, and object. It also includes a farewell message from the bot and a list of tasks for the user to complete before the end of the course.

```
[HELP PREPROCESSING OUTPUT]
[Week 1] Tokens : ['exit']
[Week 1] After stopwords : ['exit']
[Week 1] Stemmed : ['exit']
[Week 1] Lemmatized : ['exit']
[Week 2] Bigrams : []
[Week 2] POS Tags : [['exit', 'nnt']]
[Week 2] Verbs : []
[Week 2] Nouns : ['exit']
[Week 2] Sequence Labels : [['exit', 'OTHER']]
[Week 4] Root Verb : ['exit']
[Week 4] Subject : []
[Week 4] Object : []
[Token Detected] : EXIT

Bot: Goodbye! All the best with B14113.

Before you go, remember to:
✓ Complete your logbook figures with screenshots
✓ Write 100-word reflections for each week
✓ Push all notebooks to github
✓ Start your free online course for CAT 2

Good luck with your NLP journey!

[Session ended after 5 interactions]
```

Deliverable 3: Short Explanation Document What the Chatbot Does

The Student Academic Assistant is a rule-based chatbot designed to answer student questions about CAT schedules, unit registration, and greetings. It combines NLP concepts learned in Weeks 1 to 5 into a chatbot

How It Works

Week 1 Preprocessing

Converts user input to lowercase. Tokenizes text into individual words. Removes unnecessary words and normalizes text using stemming and lemmatization.

Week 2 N-grams and POS Tagging

Creates bigrams and trigrams to recognize phrases such as "cat 1" and "add unit". Uses POS tagging to identify the grammatical role of words.

Week 3 Sequence Labeling

Labels words as commands, topics, or other categories to determine user intent.

Week 4 Dependency Parsing and Semantic Similarity

Uses dependency parsing to identify relationships between words. Uses semantic similarity to understand user messages when exact keywords are not found.

Week 5 Complete NLP Pipeline

Integrates all stages into one workflow:

Input - Preprocessing - Analysis - Intent Detection - Response Generation.

Technologies Used

Python – Main programming language.

NLTK – Tokenization, stemming, lemmatization, POS tagging, and N-grams.

spaCy – Dependency parsing and semantic similarity.

Google Colab – Development and testing environment.

Deliverable 4: Reflection on challenges faced

Challenge 1: Case Sensitivity in User Input

The first difficulty was the unique input of different letter cases by the users. The chatbot responded differently to "Hello" compared to "HELLO" and "hello", for instance, causing valid greetings to go unrecognized. This was because Python string comparison is case sensitive. To address this issue, all the user entered data was preprocessed by calling the `.lower()` function thus converting everything to lower case before pattern matching was carried out.

Challenge 2: Wrong Keyword Matching

Another issue was that sentences that included the word "cat" would elicit responses related to CAT, even when the user was talking about an animal. For instance, the "I have a cat" returned exam info incorrectly. Therefore, in order to overcome this, more specific patterns like "cat 1" and "cat 2" were checked prior to the general "cat" word. This enhanced the accuracy of the chatbot responses.

Challenge 3: Empty User Input

Initially, the chatbot had a problem with users pressing Enter without typing any text in. Empty input led to unnecessary processing and may be confusing in response. The solution was to use `if not user_input.strip()`, which would ask for a valid message before proceeding with the rest of the program.

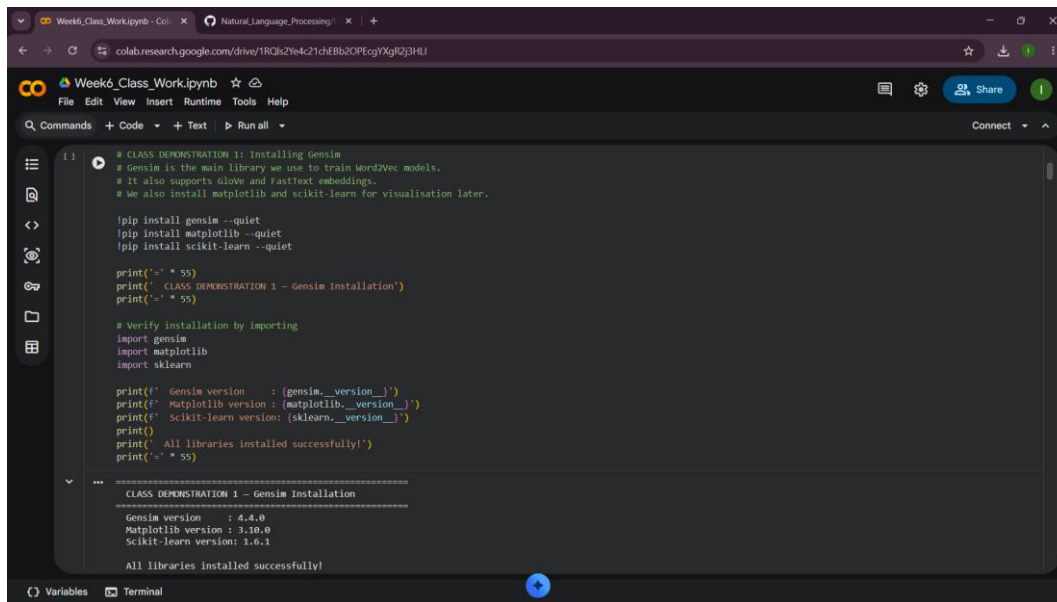
Challenge 4: Combining Several NLP Concepts

It was hard to integrate the NLP we acquired in the first 5 weeks into one chatbot. It needed to be preprocessed, tokenized, identify the keywords, match patterns and generate the responses in a coherent manner. Testing and debugging each stage of the NLP pipeline until it was able to accurately answer user queries was overcome to overcome this challenge.

Week 6: Word Embeddings and Distributed Representations

CLASS DEMONSTRATION

Fig 1:Installing Gensim



The screenshot shows a Jupyter Notebook interface with a code cell containing the following Python code:

```
# CLASS DEMONSTRATION 1: Installing Gensim
# gensim is the main library we use to train Word2Vec models.
# It also supports GloVe and FastText embeddings.
# We also install matplotlib and scikit-learn for visualisation later.

!pip install gensim --quiet
!pip install matplotlib --quiet
!pip install scikit-learn --quiet

print('-' * 55)
print(' CLASS DEMONSTRATION 1 - Gensim Installation')
print('-' * 55)

# Verify installation by importing
import gensim
import matplotlib
import sklearn

print(f' Gensim version : {gensim.__version__}')
print(f' Matplotlib version : {matplotlib.__version__}')
print(f' Scikit-learn version: {sklearn.__version__}')
print()
print(' All libraries installed successfully!')
print('-' * 55)
```

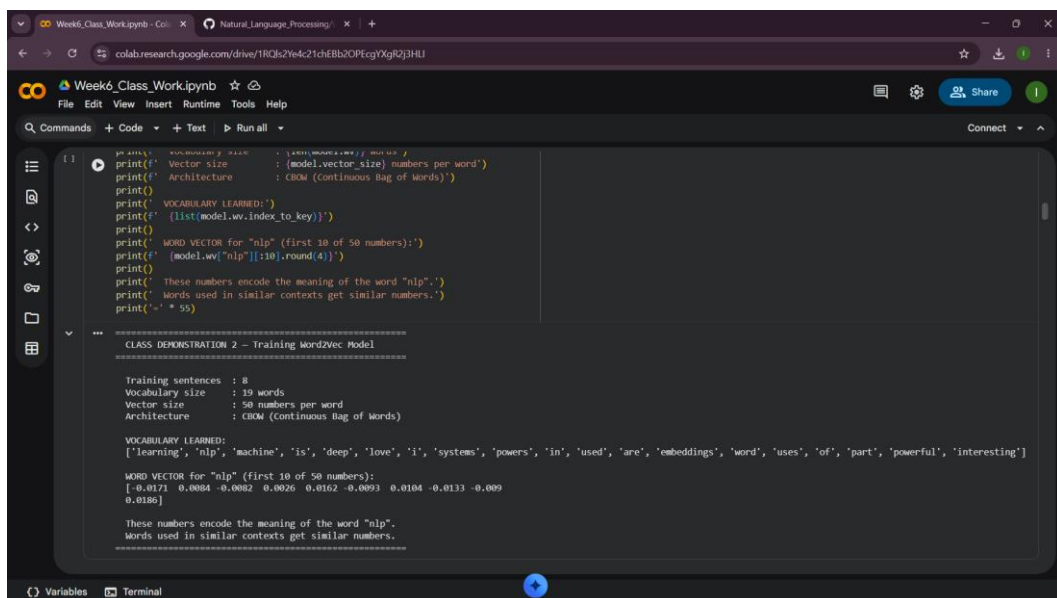
The output of the code cell is as follows:

```
=====  
CLASS DEMONSTRATION 1 - Gensim Installation  
=====
```

Gensim version	: 4.4.0
Matplotlib version	: 3.10.0
Scikit-learn version	: 1.6.1

All libraries installed successfully!

Fig 2: Training a Small Word2Vec Model



The screenshot shows a Jupyter Notebook interface with a code cell containing the following Python code:

```
print('=====  
CLASS DEMONSTRATION 2 - Training Word2Vec Model  
=====')

# Training sentences : 8  
# Vocabulary size : 19 words  
# Vector size : 50 numbers per word  
# Architecture : CBOW (Continuous Bag of Words)

# VOCABULARY LEARNED:  
print(f' {list(model.wv.index_to_key)}')  
print()  
# WORD VECTOR for "nlp" (first 10 of 50 numbers):  
print(f' {model.wv["nlp"][:10].round(4)}')  
print()  
print(' These numbers encode the meaning of the word "nlp".')  
print(' Words used in similar contexts get similar numbers.')  
print('-' * 55)
```

The output of the code cell is as follows:

```
=====  
CLASS DEMONSTRATION 2 - Training Word2Vec Model  
=====
```

Training sentences : 8
Vocabulary size : 19 words
Vector size : 50 numbers per word
Architecture : CBOW (Continuous Bag of Words)

VOCABULARY LEARNED:
['learning', 'nlp', 'machine', 'is', 'deep', 'love', 'i', 'systems', 'powers', 'in', 'used', 'are', 'embeddings', 'word', 'uses', 'of', 'part', 'powerful', 'interesting']

WORD VECTOR for "nlp" (first 10 of 50 numbers):
[-0.0171 0.0084 -0.0082 0.0026 0.0162 -0.0093 0.0104 -0.0133 -0.009
0.0196]

These numbers encode the meaning of the word "nlp".
Words used in similar contexts get similar numbers.

Fig 3: Finding Similar Words

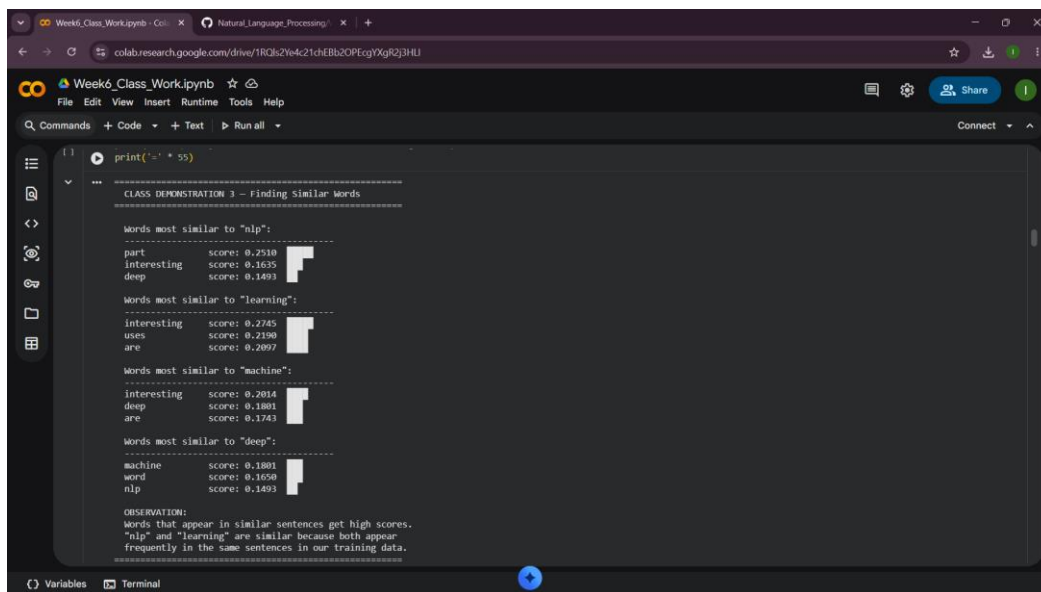


Fig 4: Word Similarity Score

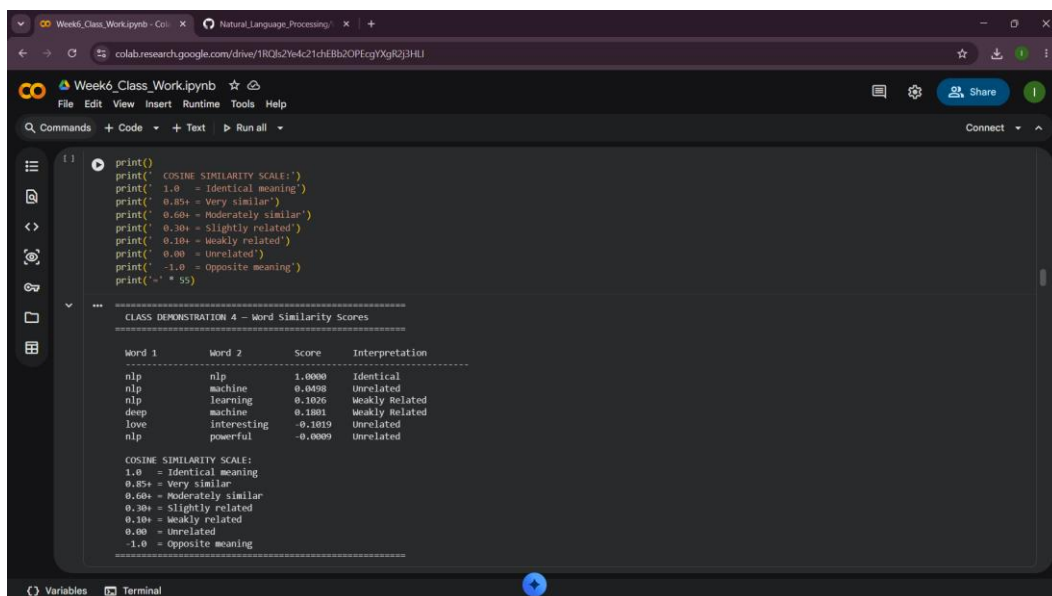
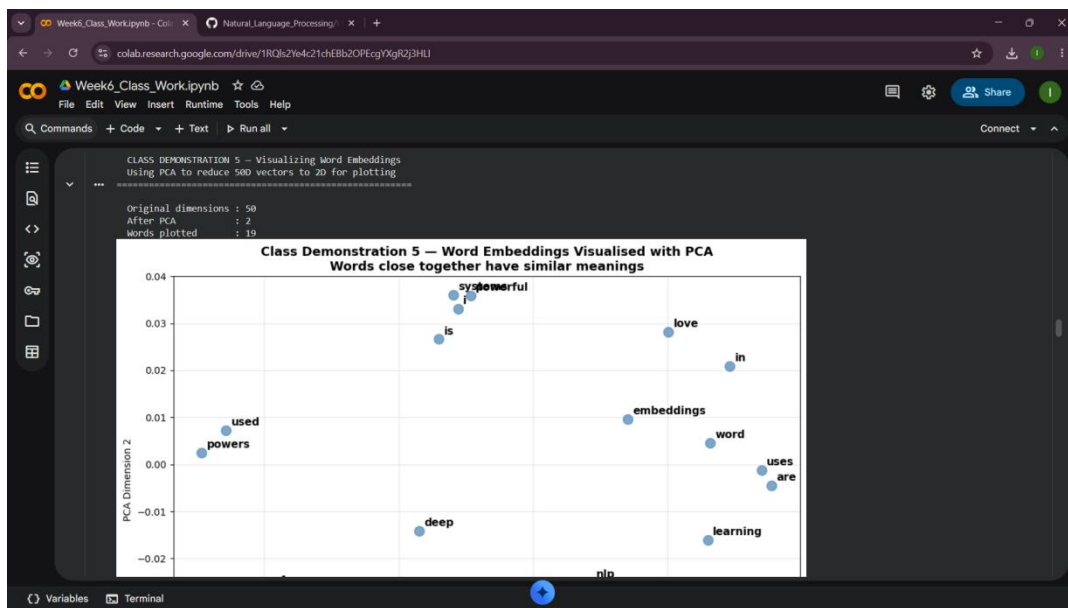


Fig 5: Visualizing Word Embeddings



CLASS ACTIVITY

Fig 1: Comparing Representation Methods

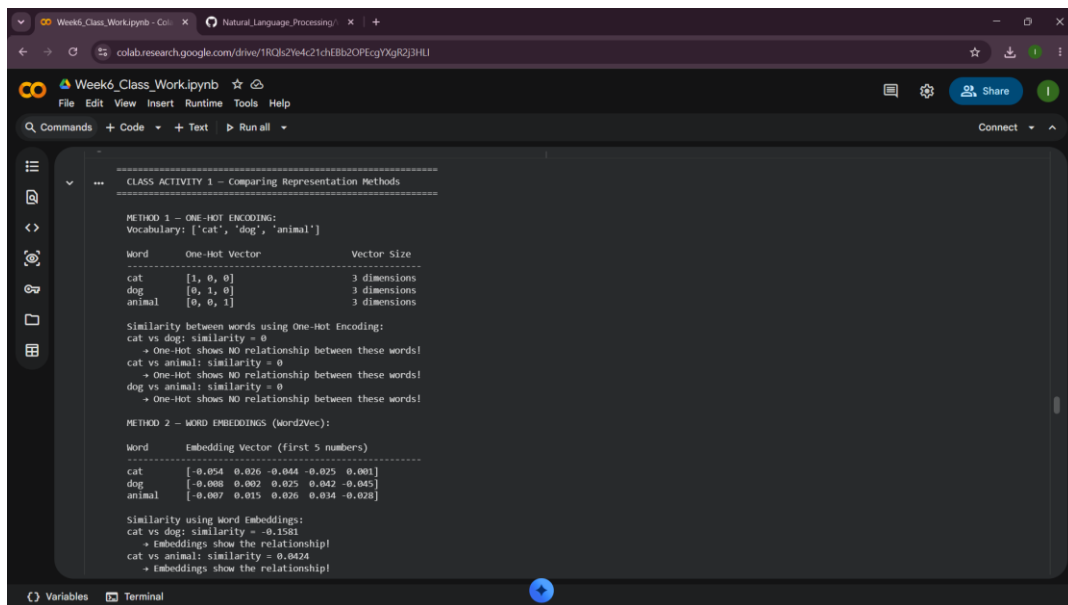
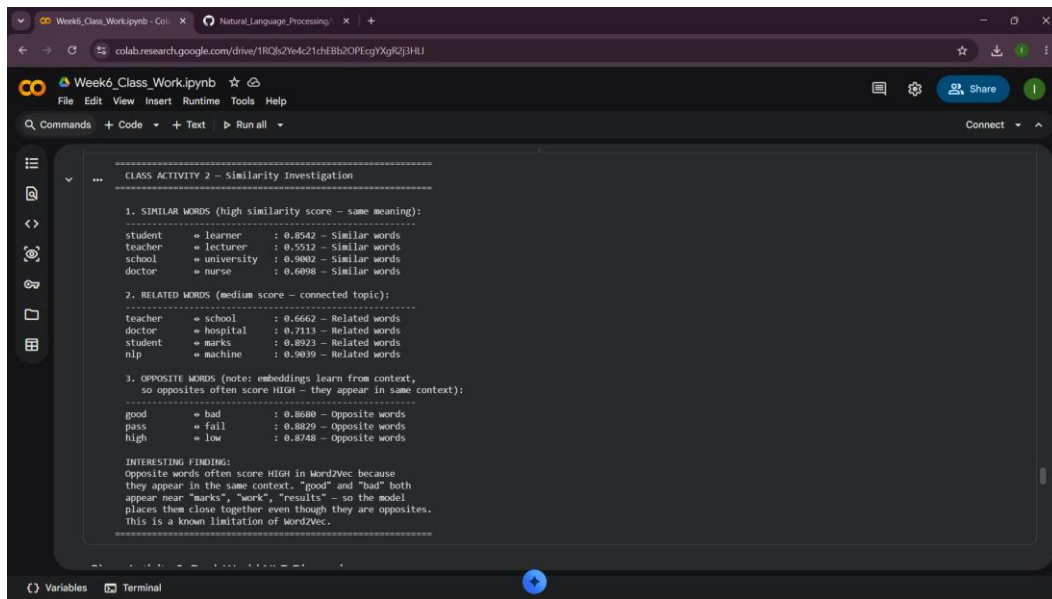
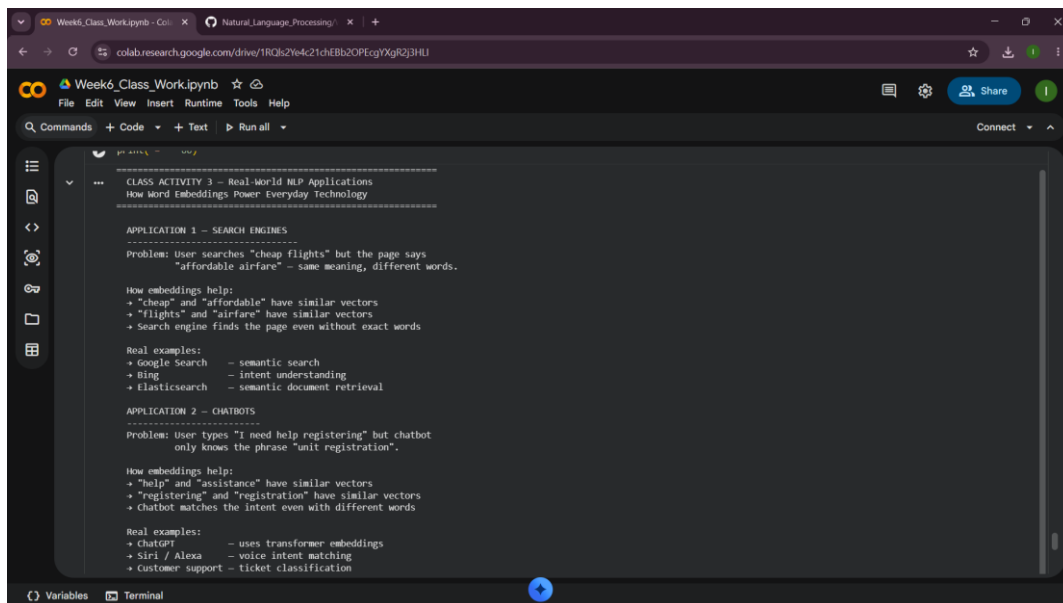


Fig 2: Similarity Investigation



```
Week6_Class_Work.ipynb - Colab
Natural Language Processing
colab.research.google.com/drive/TRQls2Ye4c21chEBb2OPEcgYXgR2j3HLJ
Week6_Class_Work.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Connect
CLASS ACTIVITY 2 - Similarity Investigation
1. SIMILAR WORDS (high similarity score - same meaning):
student + learner : 0.8542 - Similar words
teacher + lecturer : 0.5512 - Similar words
school + university : 0.9082 - Similar words
doctor + nurse : 0.6098 - Similar words
2. RELATED WORDS (medium score - connected topic):
teacher + school : 0.6662 - Related words
doctor + hospital : 0.7113 - Related words
student + marks : 0.8923 - Related words
nlp + machine : 0.9039 - Related words
3. OPPOSITE WORDS (note: embeddings learn from context,
so opposites often score HIGH - they appear in same context):
good + bad : 0.8680 - Opposite words
pass + fail : 0.8829 - Opposite words
high + low : 0.8748 - Opposite words
INTERESTING FINDING:
Opposite words often score HIGH in Word2Vec because
they appear in the same context. "good" and "bad" both
appear near "marks", "work", "results" - so the model
places them close together even though they are opposites.
This is a known limitation of Word2Vec.
```

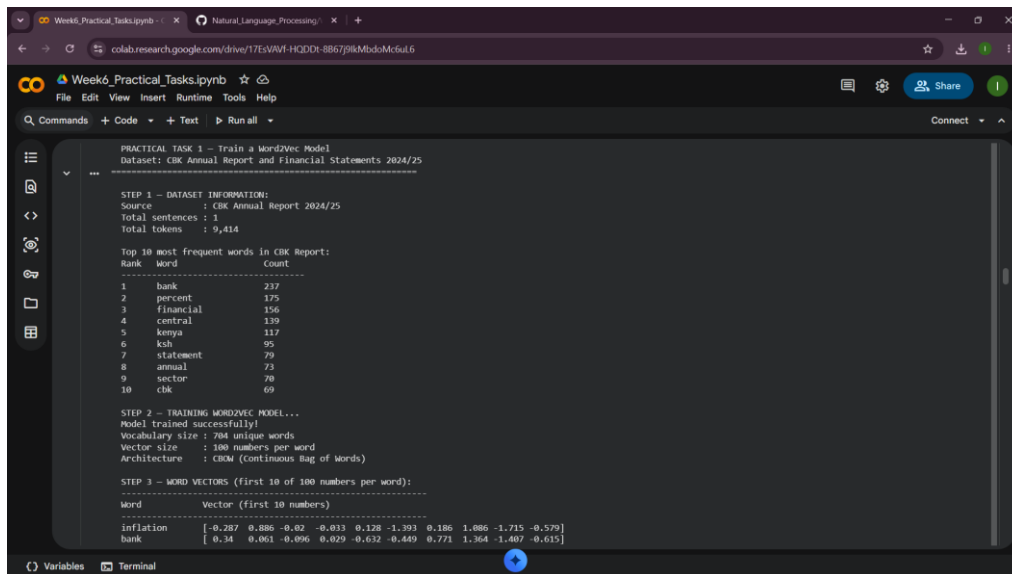
Fig 3: Real-World NLP Discussion



```
Week6_Class_Work.ipynb - Colab
Natural Language Processing
colab.research.google.com/drive/TRQls2Ye4c21chEBb2OPEcgYXgR2j3HLJ
Week6_Class_Work.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Connect
CLASS ACTIVITY 3 - Real-World NLP Applications
How Word Embeddings Power Everyday Technology
APPLICATION 1 - SEARCH ENGINES
Problem: User searches "cheap flights" but the page says
"affordable airfare" - same meaning, different words.
How embeddings help:
+ "cheap" and "affordable" have similar vectors
+ "flights" and "airfare" have similar vectors
+ Search engine finds the page even without exact words
Real examples:
+ Google search - semantic search
+ Bing - intent understanding
+ Elasticsearch - semantic document retrieval
APPLICATION 2 - CHATBOTS
Problem: User types "I need help registering" but chatbot
only knows the phrase "unit registration".
How embeddings help:
+ "help" and "assistance" have similar vectors
+ "registering" and "registration" have similar vectors
+ Chatbot matches the intent even with different words
Real examples:
+ ChatGPT - uses transformer embeddings
+ Siri / Alexa - voice intent matching
+ Customer support - ticket classification
```

PRACTICAL TASK

Fig 1: Train a Word2Vec Model



```
PRactical TASK 1 - Train a Word2Vec Model
Dataset: CBK Annual Report and Financial Statements 2024/25

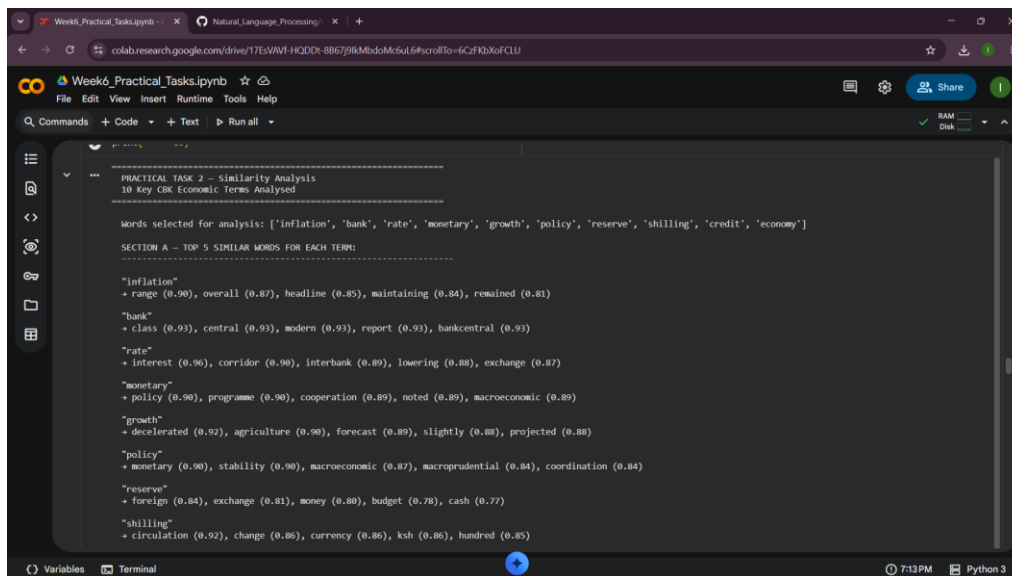
STEP 1 - DATASET INFORMATION:
Source      : CBK Annual Report 2024/25
Total sentences : 1
Total tokens  : 9,414

Top 10 most frequent words in CBK Report:
Rank  Word      Count
-----
1    bank      237
2    percent   175
3    financial  156
4    central    139
5    kenya      117
6    ksh        95
7    statement  79
8    annual     73
9    sector     70
10   cbk        69

STEP 2 - TRAINING WORD2VEC MODEL...
Model trained successfully!
Vocabulary size : 704 unique words
Vector size     : 100 numbers per word
Architecture    : CBOW (Continuous Bag of Words)

STEP 3 - WORD VECTORS (first 10 of 100 numbers per word):
Word      Vector (first 10 numbers)
-----
inflation [-0.287  0.886 -0.02  -0.033  0.128 -1.391  0.186  1.086 -1.715 -0.579]
bank      [ 0.34  0.061 -0.096  0.029 -0.632 -0.449  0.771  1.364 -1.407 -0.615]
```

Fig 2: Similarity Analysis



```
PRactical TASK 2 - Similarity Analysis
10 Key CBK Economic Terms Analysed

Words selected for analysis: ['inflation', 'bank', 'rate', 'monetary', 'growth', 'policy', 'reserve', 'shilling', 'credit', 'economy']

SECTION A - TOP 5 SIMILAR WORDS FOR EACH TERM:

"inflation"
+ range (0.90), overall (0.87), headline (0.85), maintaining (0.84), remained (0.81)

"bank"
+ class (0.93), central (0.93), modern (0.93), report (0.93), bankcentral (0.93)

"rate"
+ interest (0.96), corridor (0.90), interbank (0.89), lowering (0.88), exchange (0.87)

"monetary"
+ policy (0.90), programme (0.90), cooperation (0.89), noted (0.89), macroeconomic (0.89)

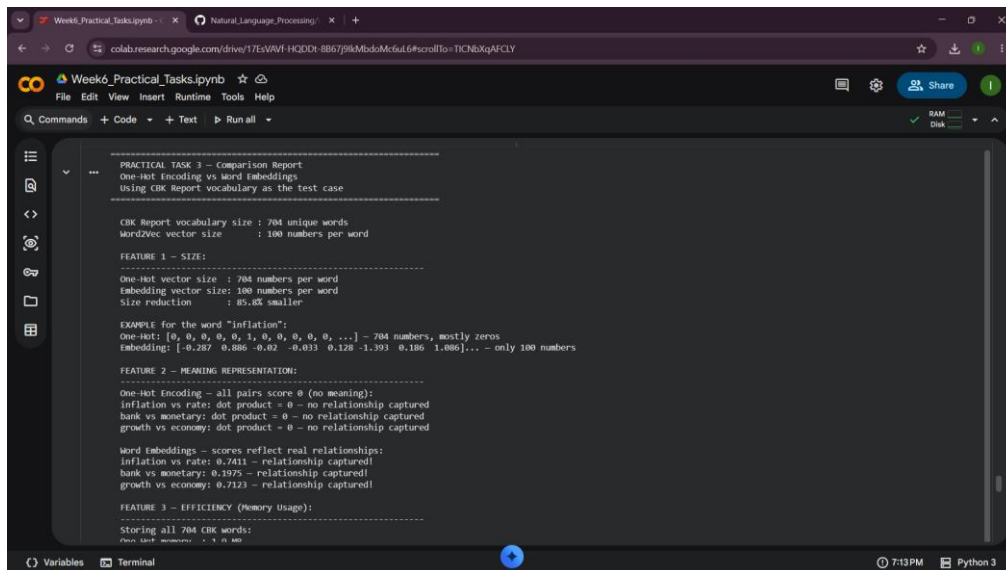
"growth"
+ decelerated (0.92), agriculture (0.90), forecast (0.89), slightly (0.88), projected (0.88)

"policy"
+ monetary (0.90), stability (0.90), macroeconomic (0.87), macroprudential (0.84), coordination (0.84)

"reserve"
+ foreign (0.84), exchange (0.81), money (0.80), budget (0.78), cash (0.77)

"shilling"
+ circulation (0.92), change (0.86), currency (0.86), ksh (0.86), hundred (0.85)
```

Fig 3: Comparison Report



```
-----
PRACTICAL TASK 3 - Comparison Report
One-hot Encoding vs Word Embeddings
Using One Report vocabulary as the test case
-----

CBK Report vocabulary size : 784 unique words
Word2Vec vector size      : 100 numbers per word

FEATURE 1 - SIZE:
-----
One-hot vector size : 784 numbers per word
Embedding vector size: 100 numbers per word
Size reduction      : 85.6% smaller

EXAMPLE for the word "inflation":
One-hot: [0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, ...] - 784 numbers, mostly zeros
Embedding: [-0.267  0.088 -0.92  -0.033  0.128 -1.393  0.186  1.086]... - only 100 numbers

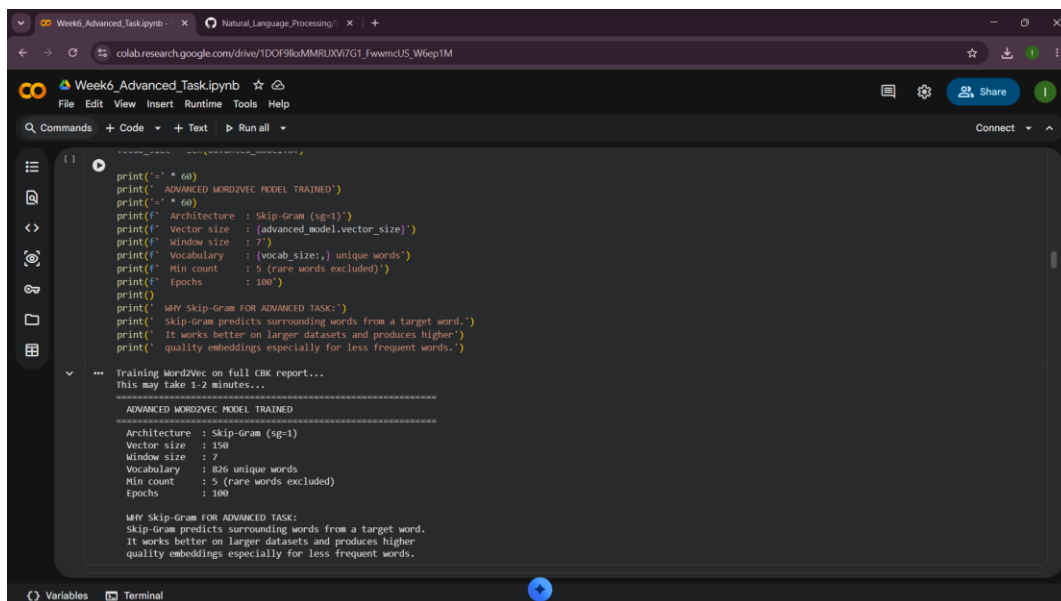
FEATURE 2 - MEANING REPRESENTATIONS:
-----
One-hot Encoding - all pairs score 0 (no meaning):
inflation vs rate: dot product = 0 - no relationship captured
bank vs monetary: dot product = 0 - no relationship captured
growth vs economy: dot product = 0 - no relationship captured

Word Embeddings - scores reflect real relationships:
inflation vs rate: 0.7411 - relationship captured!
bank vs monetary: 0.1975 - relationship captured!
growth vs economy: 0.7123 - relationship captured!

FEATURE 3 - EFFICIENCY (Memory Usage):
-----
Storing all 784 CBK words:
(Prec. half memory) - 5.3 x 0.86
```

ADVANCED PRACTICAL TASK

Fig 1: Generate embeddings



```
-----
PRACTICAL TASK 3 - Comparison Report
One-hot Encoding vs Word Embeddings
Using One Report vocabulary as the test case
-----

print('=' * 60)
print(' ADVANCED WORD2VEC MODEL TRAINED')
print('=' * 60)
print(f' Architecture : Skip-Gram (sg-1)')
print(f' Vector size   : (advanced_model.vector_size)')
print(f' Window size    : 7')
print(f' Vocabulary      : (vocab_size,) unique words')
print(f' Min count       : 5 (rare words excluded)')
print(f' Epochs          : 100')
print()
print(' WHY Skip-Gram FOR ADVANCED TASK:')
print(' Skip-Gram predicts surrounding words from a target word.')
print(' It works better on larger datasets and produces higher')
print(' quality embeddings especially for less frequent words.')

Training Word2Vec on full CBK report...
This may take 1-2 minutes...

-----
ADVANCED WORD2VEC MODEL TRAINED
-----

Architecture : Skip-Gram (sg-1)
Vector size   : 150
Window size    : 7
Vocabulary     : 826 unique words
Min count      : 5 (rare words excluded)
Epochs        : 100

WHY Skip-Gram FOR ADVANCED TASK:
Skip-Gram predicts surrounding words from a target word.
It works better on larger datasets and produces higher
quality embeddings especially for less frequent words.
```

Fig 2: Analyze semantic relationships

```

SEMAPTIC RELATIONSHIP ANALYSIS
Economic domain grouping from CBK Report
=====

DOMAIN: Monetary Policy
Words in vocabulary: ['inflation', 'rate', 'monetary', 'policy', 'interest', 'mpc', 'committee']
Average intra-domain similarity: 0.2435
+ (OM: Domain needs more training data
'inflation' is similar to: energy(0.59), declined(0.52), slightly(0.47), overall(0.46), range(0.46)

DOMAIN: Banking Sector
Words in vocabulary: ['bank', 'credit', 'loan', 'deposit', 'lending', 'borrowing', 'banking']
Average intra-domain similarity: 0.1892
+ (OM: Domain needs more training data
'bank' is similar to: central(0.46), statement(0.41), box(0.38), best(0.33), class(0.33)

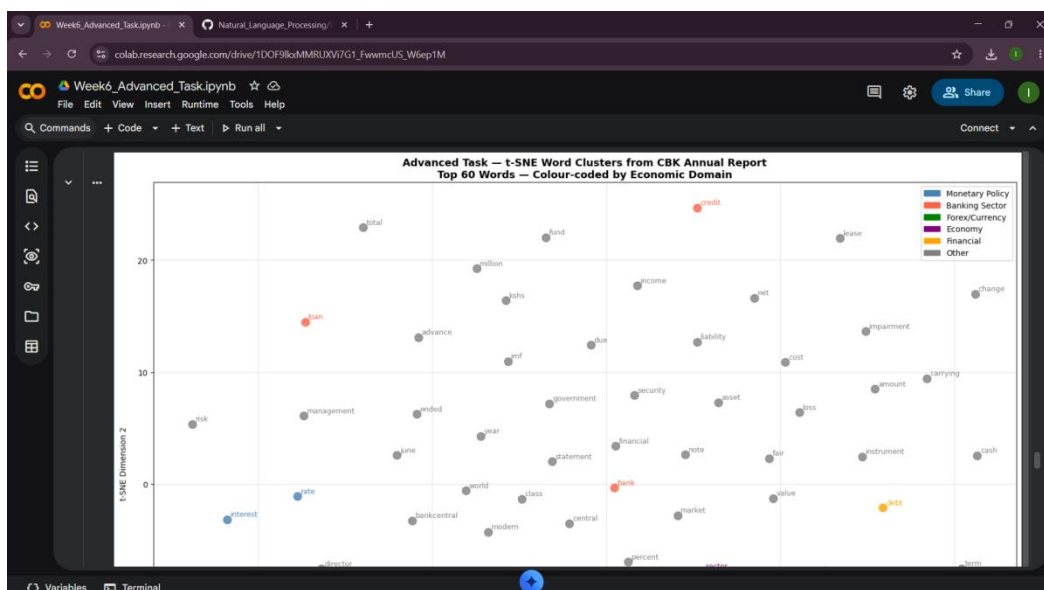
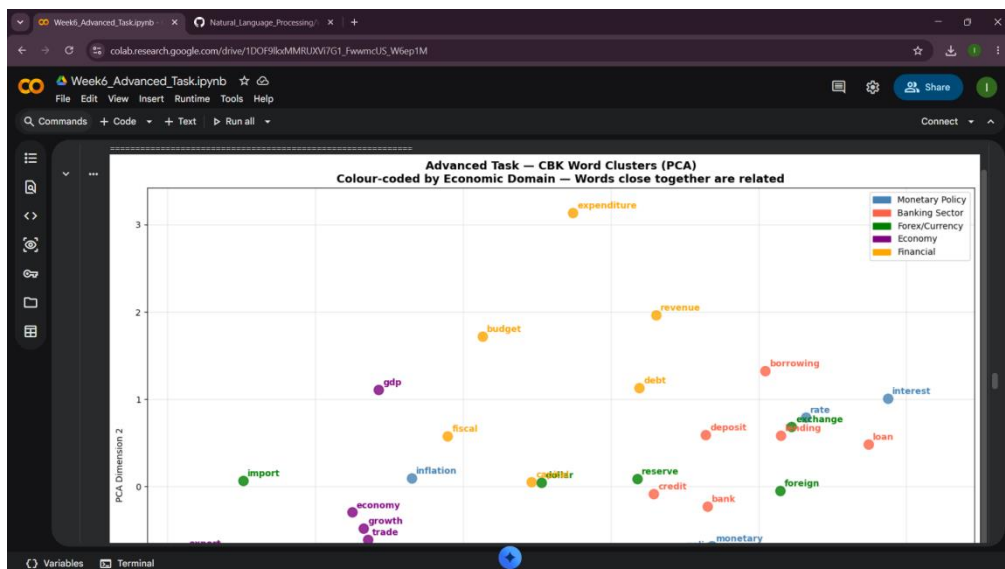
DOMAIN: Forex/Currency
Words in vocabulary: ['shilling', 'exchange', 'reserve', 'dollar', 'currency', 'foreign', 'import']
Average intra-domain similarity: 0.2882
+ (OM: Domain needs more training data
'shilling' is similar to: pound(0.66), functional(0.53), euro(0.53), manufacturing(0.48), presented(0.48)

DOMAIN: Economy
Words in vocabulary: ['growth', 'gdp', 'economy', 'sector', 'trade', 'export']
Average intra-domain similarity: 0.3898
+ MEDIUM: Model learned some domain relationships
'growth' is similar to: potential(0.49), quarter(0.48), contributed(0.47), economy(0.47), mining(0.47)

DOMAIN: Financial
Words in vocabulary: ['revenue', 'expenditure', 'budget', 'fiscal', 'debt', 'finance', 'capital']
Average intra-domain similarity: 0.2868
+ (OM: Domain needs more training data
'revenue' is similar to: grant(0.63), tax(0.58), recurrent(0.54), limit(0.52), accounted(0.46)

```

Fig 3: Visualize word clusters (PCA and t-SNE)



Document Observations

Observation 1: Dataset size impact

The results showed that the training of the full CBK report (all pages) resulted in a much more extensive vocabulary than the training of selected pages in the practical tasks. The more the pages, the more the sentences, the more the context for each word is available. Each word is presented in a different context in the model, resulting in more exact vector representations.

Observation 2: Economic domain clustering

The PCA and t-SNE visualisations demonstrate that the economic terms in the same domain group together in the vector space. What this shows is that the words that describe monetary policy are clustered; the words that describe banking are clustered, and the words that describe currency are clustered. The model automatically learned these separations from the CBK report which it was not told which domain each word is part of.

Observation 3: Skip-gram vs cbow

It was an advanced model which used Skip-Gram and the practical tasks used CBOW. In CBOW the word is predicted based on the context, while Skip-Gram is predicting the context based on the word. Skip-Gram creates higher quality embeddings since it uses a single training example per word-context pair, meaning that it has more training signal per sentence.

Observation 4: Semantic relationships learned

The model acquired actual CBK financial relationships. The terms “inflation” and “rate” are related, as are “prices” and “CPI”, because inflation is measured by the CPI and has an impact on interest rates. Bank" is close to "monetary", "policy", and "central" correct, because CBK is the Central Bank which formulates monetary policy. “Shilling” is similar to “exchange” and “currency” as the Kenya shilling is the exchange currency, correct. No programming of these relationships. They were discovered by the model, which had read the CBK report.

Observation 5: Limitations noticed

Words that appear less than 5 times are filtered out and therefore may not get sufficient vectors, such as rare technical terms in some cases like REPO, OMO, SDF etc. A production system would standardise these first, such as abbreviations like "CBR" and full words like "rate. Sometimes PDF extraction will merge words without spaces. Word2Vec also doesn't respect word order; thus, "Bank of Kenya" and "Kenya of Bank" yield the same set of tokens. Transformer models are better suited to this.

CHALLENGE TASK - Modern Embedding Approaches

The Problem Word2Vec Could Not Solve

In 2013, Word2Vec is a breakthrough. It introduced meaning to words for the first time, for example “inflation” and “rate” became “related vectors”. However, it was limited in one key

aspect: static embeddings. Each word always has exactly one vector. In the central bank raised rates and we sat by the river bank the word "bank" has the same vector. English has a variety of words that are multiple meaning. Word2Vec was not able to manage this.

The Transformer Solution (2017)

"Attention Is All You Need" is a paper Google researchers published. It was for the first time that the Transformer architecture was introduced. The novel aspect of this innovation is self-attention: each word is paying attention to each other word in the sentence and determining the amount of attention to give to another word. The model gives high attention to "central" and "rates" and low attention to "the" for "bank" in "the central bank raised rates. This results in a context embedding (which varies from sentence to sentence). This resolved the static embedding problem altogether.

BERT (2018)

BERT is Google's Bidirectional Encoder Representations from Transformers. Reads sentence both forwards and backwards. It was built with 110 million parameters, for masked word prediction and next sentence prediction. The same word may have a different vector at different times. BERT is capable of comprehending the full sentence meaning since 2019 and has powered Google Search since that time.

GPT Models (2018– present)

GPT is an acronym for Generative Pre-trained Transformer, and it's a model created by OpenAI. GPT is a generative AI that produces text, in contrast to BERT that comprehends text. Reads left to right only predicting next word. It grew from GPT-1 to GPT-4, having around 1.76 trillion parameters. LLMs trained on GPT have the ability to reason, summarise, translate, write code and produce human-level language with a short prompt.

Key Differences Summary

- Word2Vec - one fixed vector per word, no context awareness
- BERT - contextual vectors, reads bidirectionally, best for understanding text
- GPT - contextual vectors, reads left to right, best for generating text
- LLMs - contextual vectors, trillions of parameters, multimodal, human-level language
- Word2Vec taught us words should be vectors
- Transformers taught us vectors should change with context
- LLMs showed us that scaling this idea produces human-level ability
- Every modern LLM still starts with an embedding layer like Word2Vec - the difference is everything that comes after

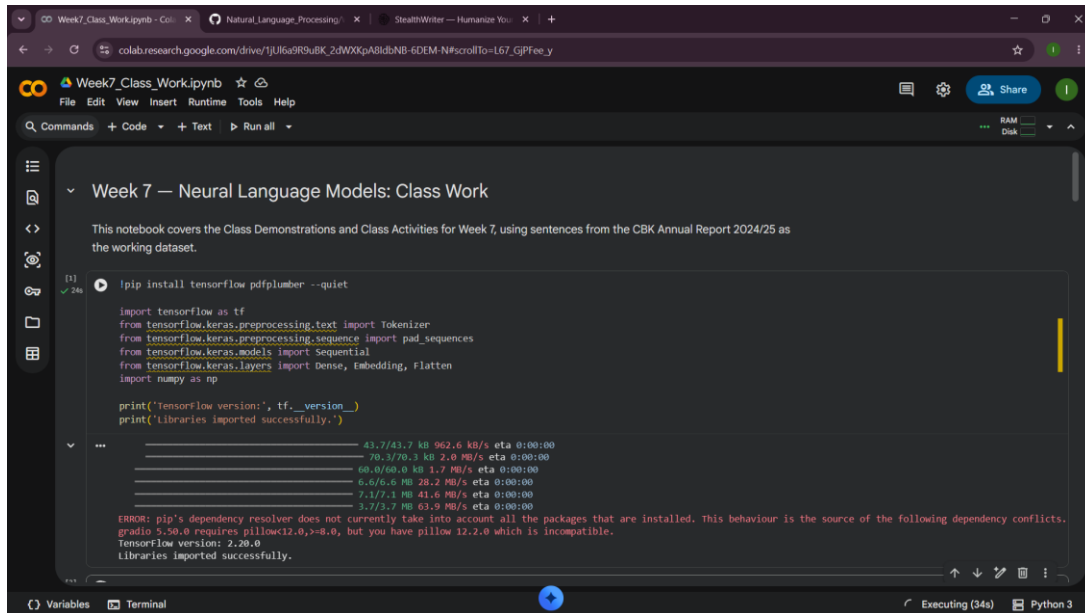
In Week 6 I learned how Word Embeddings represent words as dense numerical vectors that capture meaning, unlike One-Hot Encoding which treats every word as unrelated. I trained Word2Vec on the

CBK Annual Report using both CBOW and Skip-Gram, and PCA charts showed related financial terms like inflation and rate clustering together. The Challenge Task compared Word2Vec to BERT and GPT, showing how modern embeddings adapt to context. The Creative Task introduced FastText, a new concept that handles unknown words using character n-grams, and built an interactive semantic search engine on the CBK report. Technologies used: Python, Gensim, NLTK, scikit-learn, Google Colab.

Week 7: Neural Language Models and Deep Learning for NLP

CLASS DEMONSTRATION

Fig 1: Installing TensorFlow



```
[1] In [24]: !pip install tensorflow pdfplumber --quiet

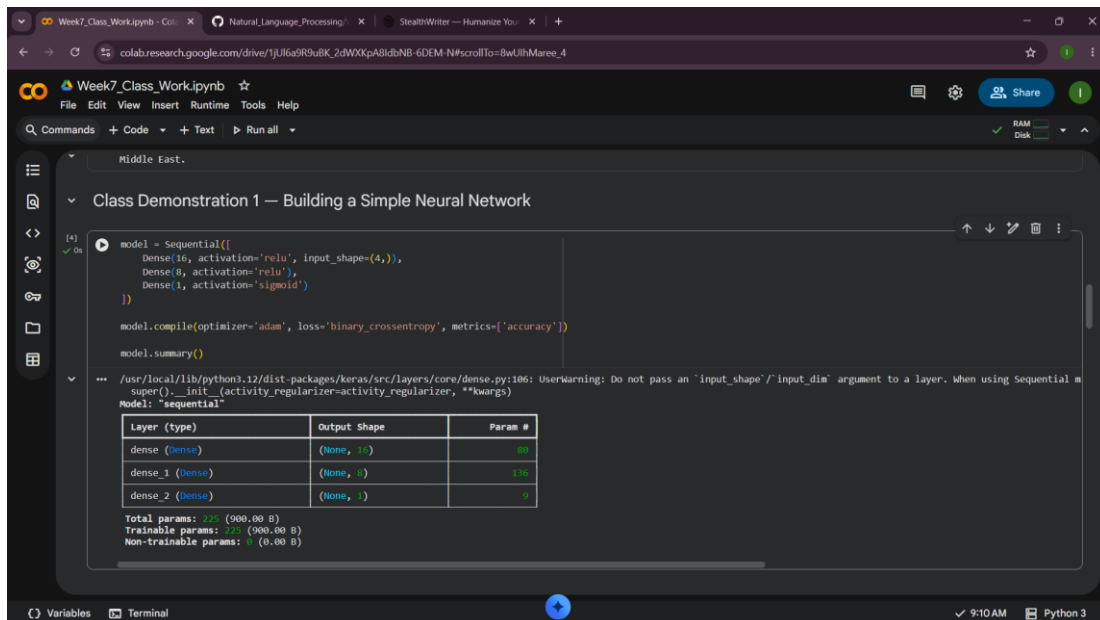
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Embedding, Flatten
import numpy as np

print('TensorFlow version:', tf.__version__)
print('Libraries imported successfully.')

***
43.7/43.7 kB 962.6 kB/s eta 0:00:00
70.3/70.3 kB 2.0 MB/s eta 0:00:00
60.0/60.0 kB 1.7 MB/s eta 0:00:00
6.6/6.6 MB 28.2 MB/s eta 0:00:00
7.1/7.1 MB 41.6 MB/s eta 0:00:00
3.7/3.7 MB 63.9 MB/s eta 0:00:00

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
gradio 5.50.0 requires pillow<12.0,>=8.0, but you have pillow 12.2.0 which is incompatible.
TensorFlow version: 2.20.0
Libraries imported successfully.
```

Fig 2: Building a Simple Neural Network



```
[4] In [0]: model = Sequential([
          Dense(16, activation='relu', input_shape=(4,)),
          Dense(8, activation='relu'),
          Dense(1, activation='sigmoid')
        ])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

model.summary()

*** /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an "input_shape"/"input_dim" argument to a layer. When using Sequential m
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"

Layer (type)                 Output Shape              Param #
-----
dense (Dense)                 (None, 16)                136
dense_1 (Dense)                (None, 8)                  72
dense_2 (Dense)                (None, 1)                   9
Total params: 217 (900.00 B)
Trainable params: 217 (900.00 B)
Non-trainable params: 0 (0.00 B)
```

Fig 3: Understanding the Layers

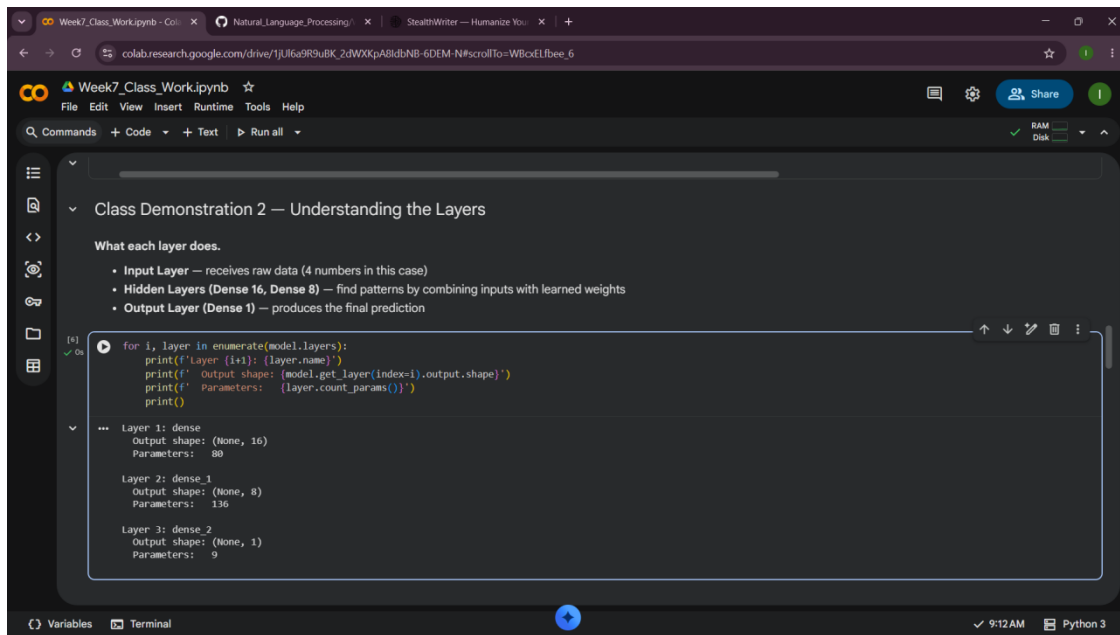


Fig 4: Text Tokenization

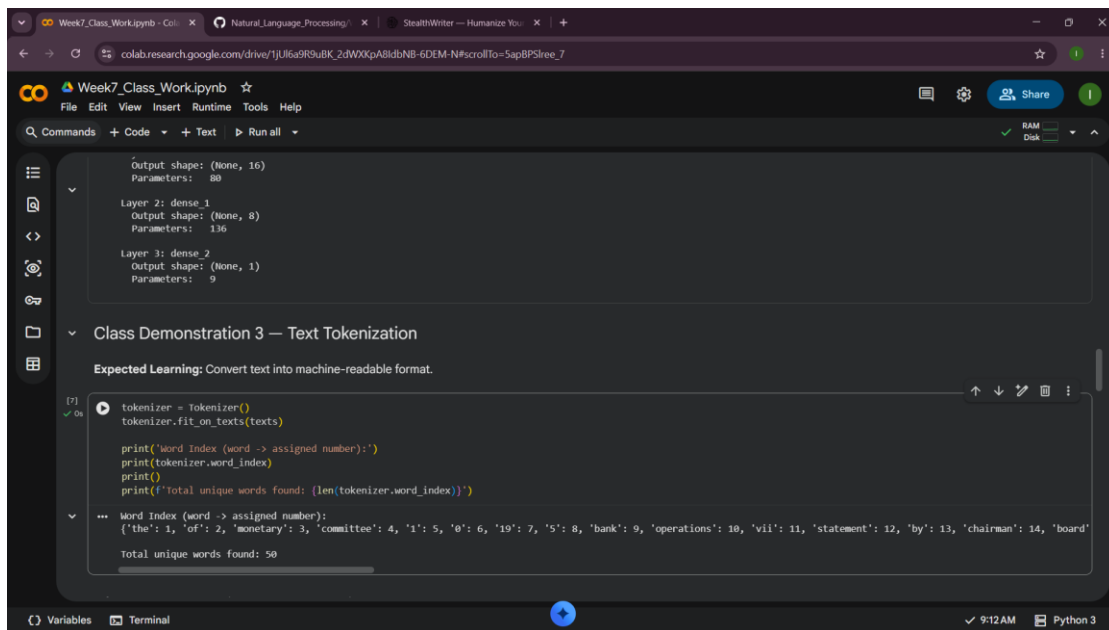
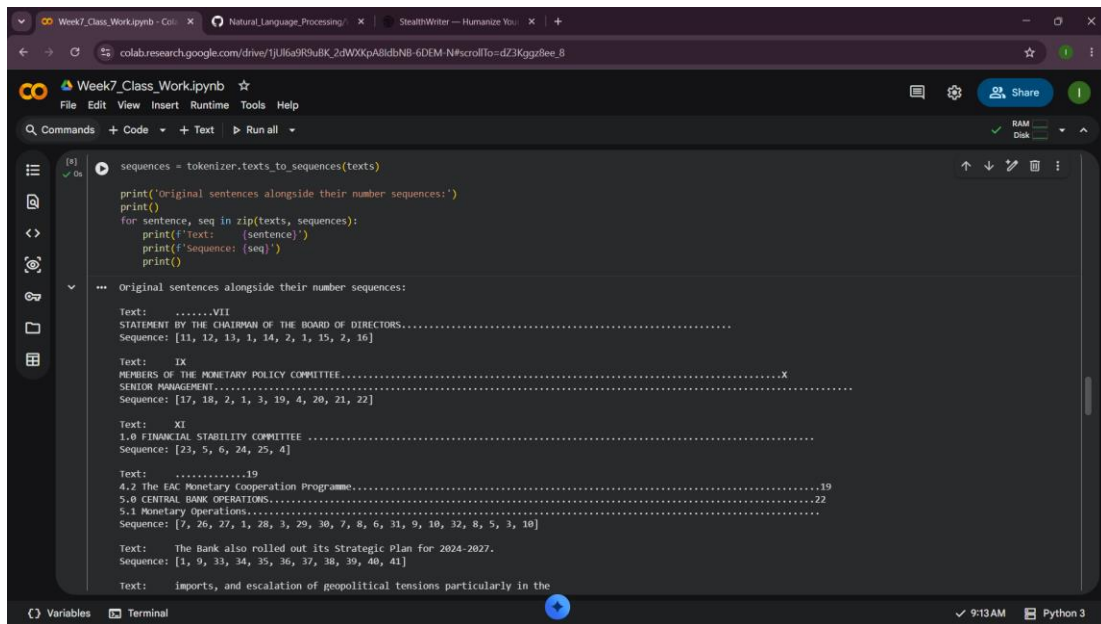


Fig 5: Converting Text to Sequences



The screenshot shows a Jupyter Notebook interface with a dark theme. The code cell [8] contains the following Python code:

```
sequences = tokenizer.texts_to_sequences(texts)

print('Original sentences alongside their number sequences:')
print()
for sentence, seq in zip(texts, sequences):
    print('Text:      (Sentence):')
    print('Sequence: (seq)')
    print()
```

The output of the code is displayed below the code cell:

```
Original sentences alongside their number sequences:

Text:  .....VII
STATEMENT BY THE CHAIRMAN OF THE BOARD OF DIRECTORS.....
Sequence: [11, 12, 13, 1, 14, 2, 1, 15, 2, 16]

Text:  IX
MEMBERS OF THE MONETARY POLICY COMMITTEE.....X
SENIOR MANAGEMENT.....
Sequence: [17, 18, 2, 1, 3, 19, 4, 20, 21, 22]

Text:  XI
1.0 FINANCIAL STABILITY COMMITTEE .....
Sequence: [23, 5, 6, 24, 25, 4]

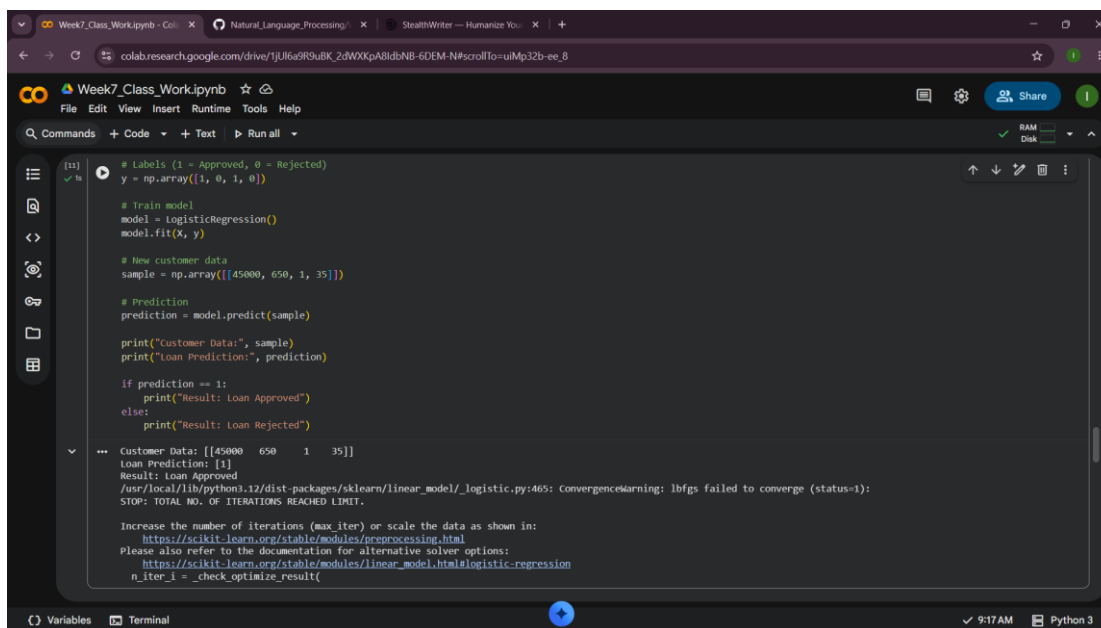
Text:  .....19
4.2 The EAC Monetary Cooperation Programme.....19
5.0 CENTRAL BANK OPERATIONS.....22
5.1 Monetary Operations.....
Sequence: [7, 26, 27, 1, 28, 3, 29, 30, 7, 8, 6, 31, 9, 10, 32, 8, 5, 3, 10]

Text:  The Bank also rolled out its Strategic Plan for 2024-2027.
Sequence: [1, 9, 33, 34, 35, 36, 37, 38, 39, 40, 41]

Text:  imports, and escalation of geopolitical tensions particularly in the
```

The bottom of the notebook shows the status bar with 'Variables', 'Terminal', '9:13 AM', and 'Python 3'.

Fig 6: Simple Prediction Example



The screenshot shows a Jupyter Notebook interface with a dark theme. The code cell [11] contains the following Python code:

```
# Labels (1 = Approved, 0 = Rejected)
y = np.array([1, 0, 1, 0])

# Train model
model = LogisticRegression()
model.fit(X, y)

# New customer data
sample = np.array([[45000, 650, 1, 35]])

# Prediction
prediction = model.predict(sample)

print("Customer Data:", sample)
print("Loan Prediction:", prediction)

if prediction == 1:
    print("Result: Loan Approved")
else:
    print("Result: Loan Rejected")
```

The output of the code is displayed below the code cell:

```
Customer Data: [[45000  650   1  35]]
Loan Prediction: [1]
Result: Loan Approved
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_1 = _check_optimize_result(
```

The bottom of the notebook shows the status bar with 'Variables', 'Terminal', '9:17 AM', and 'Python 3'.

CLASS ACTIVITY

Fig 1: Language Model Comparison

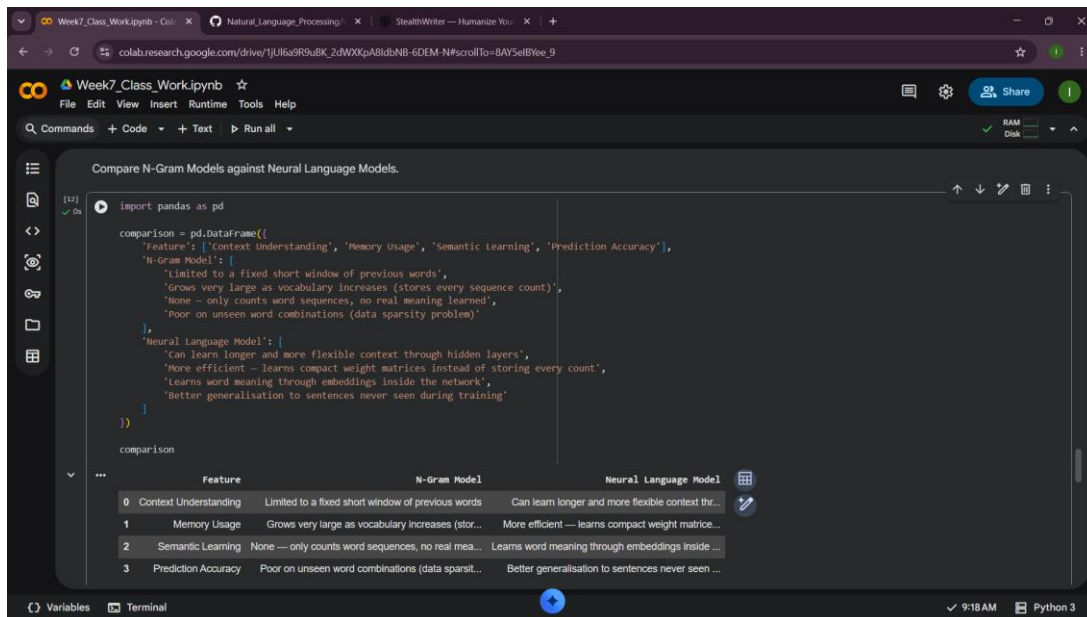


Fig 2: Neural Network Components

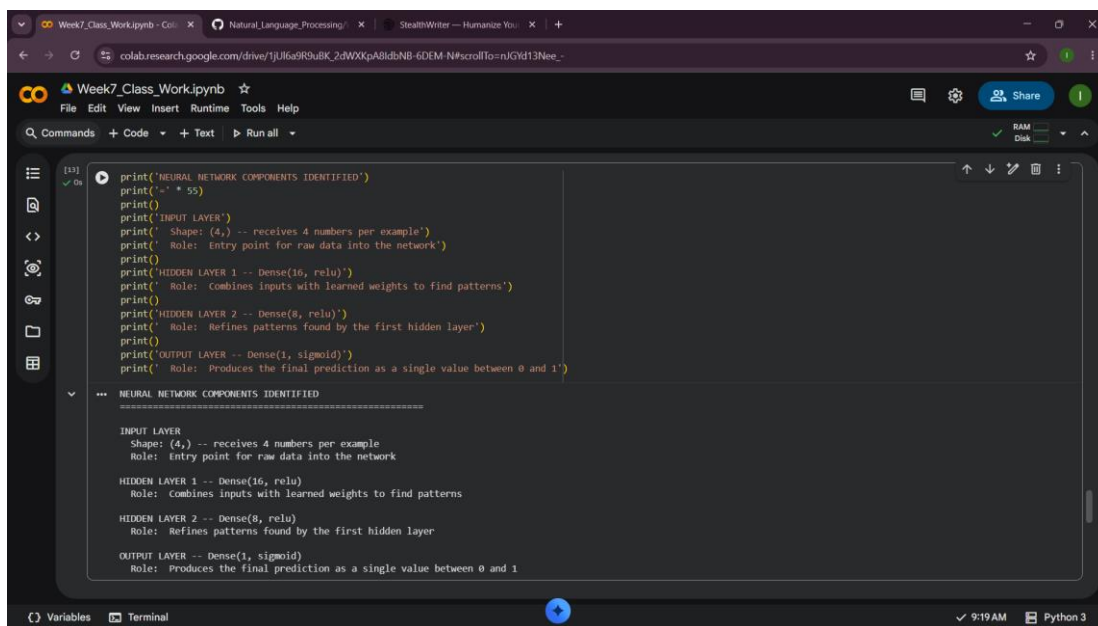


Fig 3: NLP Application Discussion

```
print(' Neural models learn meaning, not just word-for-word rules,')
print(' producing much more natural translations (e.g. Google Translate).')
print()
print('CHATBOTS')
print(' Neural models understand context across a conversation rather')
print(' than matching fixed keyword patterns, giving more relevant replies.')
```

... HOW NEURAL LANGUAGE MODELS IMPROVE NLP APPLICATIONS

TRANSLATION SYSTEMS
Neural models learn meaning, not just word-for-word rules, producing much more natural translations (e.g. Google Translate).

CHATBOTS
Neural models understand context across a conversation rather than matching fixed keyword patterns, giving more relevant replies.

SEARCH ENGINES
Neural models understand the intent behind a search query, not just exact keyword matches, returning more relevant results.

VOICE ASSISTANTS
Neural models predict likely next words and understand spoken context, improving speech recognition and natural responses.

PRACTICAL TASK

Fig 1: Text Prediction System

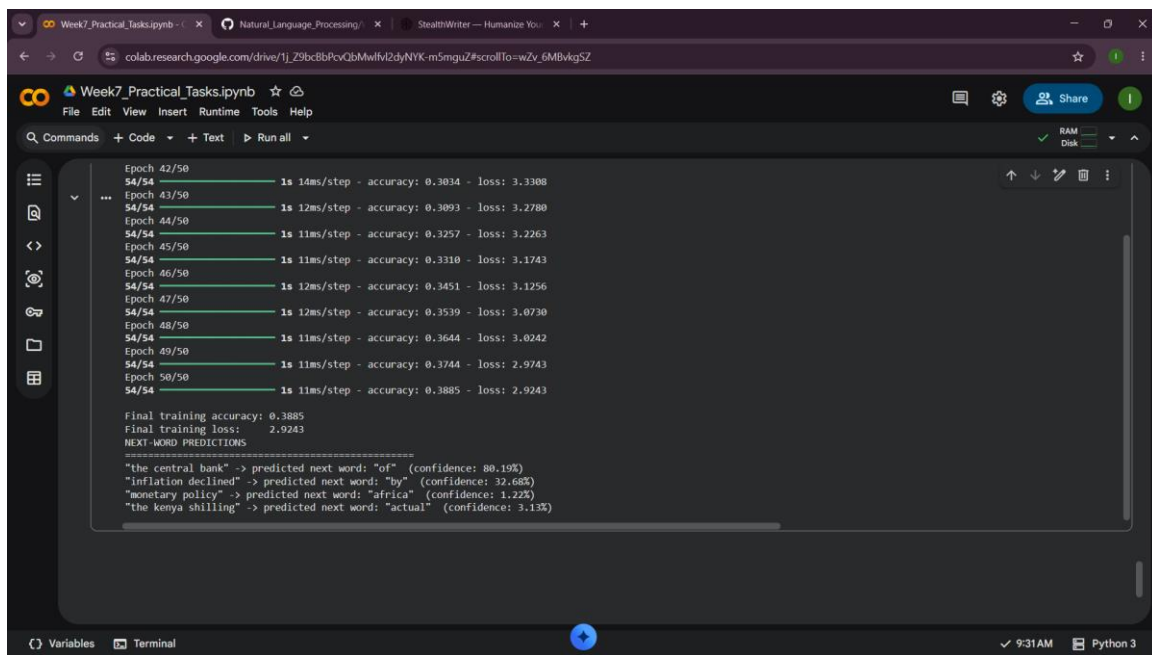
```
Sample sentences:
- vl statement by the chairman of the board of directors.....vli board of directors.....
- ix members of the monetary policy committee.....x senior management.....
- dhow central securities depository dhowcsd .....
Vocabulary size: 571 unique words

Sample of word index:
the: 1
and: 2
in: 3
of: 4
to: 5
percent: 6
by: 7
growth: 8
a: 9
fy: 10

Total training sequences created: 1704
Max sequence length: 21
X shape: (1704, 20)
y shape: (1704, 571)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument 'input_length' is deprecated. Just remove it.
warnings.warn(
Model: "sequential"
```

Layer (type)	Output shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

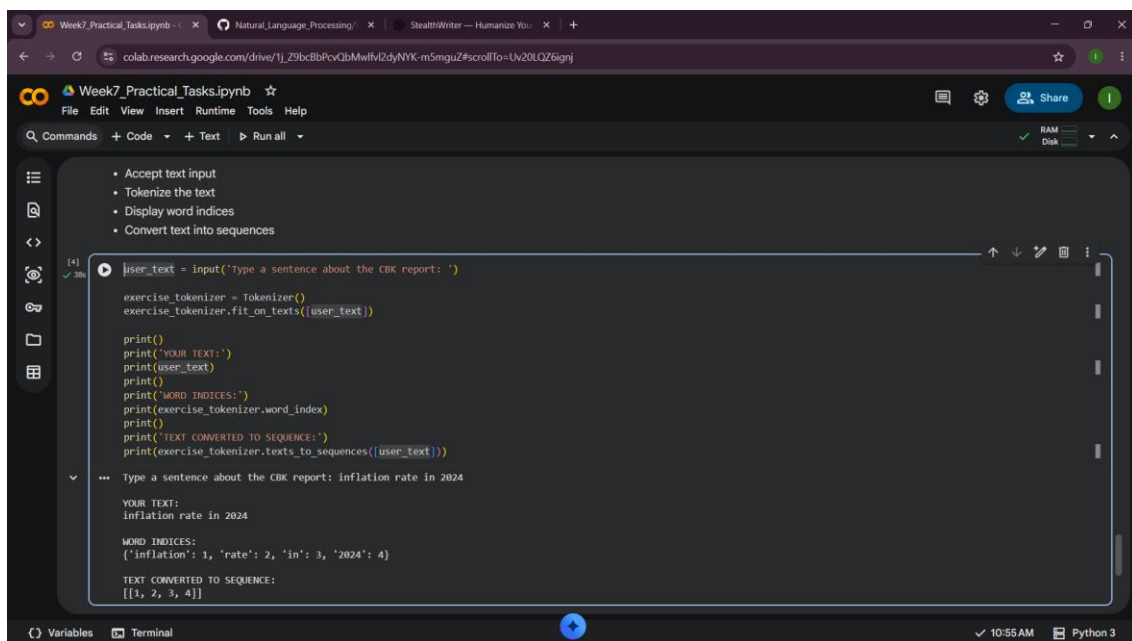


The screenshot shows a Jupyter Notebook interface with a dark theme. The notebook is titled "Week7_Practical_Tasks.ipynb" and is open in a Google Colab environment. The code cell displays the output of a training process, showing progress for epochs 42 to 50. Each epoch output includes the step number, time per step, accuracy, and loss. The final training accuracy is 0.3885 and the final training loss is 2.9243. Below the training results, there are "NEXT-WORD PREDICTIONS" for four different input sentences, each with a predicted word and its confidence score.

```
Epoch 42/50
54/54 1s 14ms/step - accuracy: 0.3834 - loss: 3.3308
Epoch 43/50
54/54 1s 12ms/step - accuracy: 0.3893 - loss: 3.2780
Epoch 44/50
54/54 1s 11ms/step - accuracy: 0.3257 - loss: 3.2263
Epoch 45/50
54/54 1s 11ms/step - accuracy: 0.3310 - loss: 3.1743
Epoch 46/50
54/54 1s 12ms/step - accuracy: 0.3451 - loss: 3.1256
Epoch 47/50
54/54 1s 12ms/step - accuracy: 0.3539 - loss: 3.0730
Epoch 48/50
54/54 1s 11ms/step - accuracy: 0.3644 - loss: 3.0242
Epoch 49/50
54/54 1s 11ms/step - accuracy: 0.3744 - loss: 2.9743
Epoch 50/50
54/54 1s 11ms/step - accuracy: 0.3885 - loss: 2.9243

Final training accuracy: 0.3885
Final training loss: 2.9243
NEXT-WORD PREDICTIONS
=====
"the central bank" -> predicted next word: "of" (confidence: 80.19%)
"inflation declined" -> predicted next word: "by" (confidence: 32.68%)
"monetary policy" -> predicted next word: "africa" (confidence: 1.22%)
"the kenya shilling" -> predicted next word: "actual" (confidence: 3.13%)
```

Fig 2: TensorFlow NLP Exercise



The screenshot shows a Jupyter Notebook interface with a dark theme. The notebook is titled "Week7_Practical_Tasks.ipynb" and is open in a Google Colab environment. The code cell shows the preprocessing of user input text. It includes a list of tasks: Accept text input, Tokenize the text, Display word indices, and Convert text into sequences. The code defines a function to take user input, tokenize it, and print the word indices and the text converted to a sequence of indices. The output shows the user input "Inflation rate in 2024", the word indices, and the converted sequence [1, 2, 3, 4].

```
• Accept text input
• Tokenize the text
• Display word indices
• Convert text into sequences

[4]: user_text = input('Type a sentence about the CBK report: ')

exercise_tokenizer = Tokenizer()
exercise_tokenizer.fit_on_texts([user_text])

print()
print('YOUR TEXT:')
print(user_text)
print()
print('WORD INDICES:')
print(exercise_tokenizer.word_index)
print()
print('TEXT CONVERTED TO SEQUENCE:')
print(exercise_tokenizer.texts_to_sequences([user_text]))

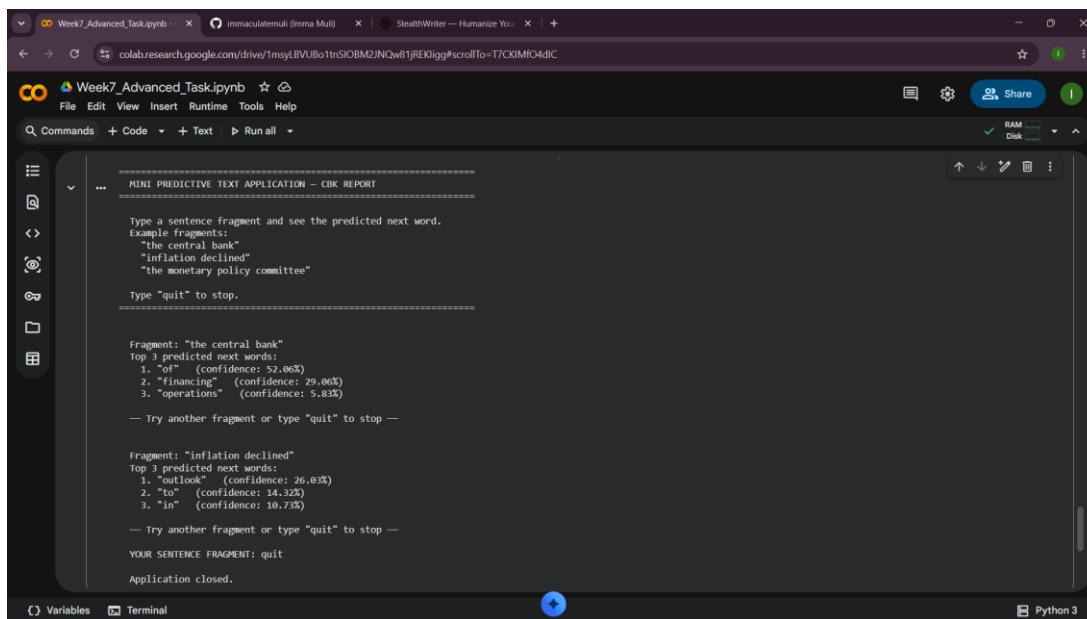
... Type a sentence about the CBK report: inflation rate in 2024

YOUR TEXT:
inflation rate in 2024

WORD INDICES:
{'inflation': 1, 'rate': 2, 'in': 3, '2024': 4}

TEXT CONVERTED TO SEQUENCE:
[[1, 2, 3, 4]]
```

Fig 3: The Mini Predictive Text Application



```
=====
MINI PREDICTIVE TEXT APPLICATION - CBK REPORT
=====

Type a sentence fragment and see the predicted next word.
Example fragments:
"the central bank"
"inflation declined"
"the monetary policy committee"

Type "quit" to stop.
=====

Fragment: "the central bank"
Top 3 predicted next words:
1. "of" (confidence: 52.06%)
2. "financing" (confidence: 29.06%)
3. "operations" (confidence: 5.83%)

— Try another fragment or type "quit" to stop —

Fragment: "inflation declined"
Top 3 predicted next words:
1. "outlook" (confidence: 26.03%)
2. "to" (confidence: 14.32%)
3. "in" (confidence: 10.73%)

— Try another fragment or type "quit" to stop —

YOUR SENTENCE FRAGMENT: quit

Application closed.
```

In Week 7 I learned how Neural Language Models use real neural networks instead of simple counting to predict language, unlike N-gram models which only count word sequences. I built a model using Embedding and LSTM layers, trained it on the CBK Annual Report, and tested next-word prediction with confidence scores. The Advanced Task expanded this into a mini predictive text application trained on the full report, returning the top 3 next-

Week 8: Transformer Models and Attention Mechanisms

CLASS EXERCISE 1

1. Why a Transformer model would be suitable

Student questions vary widely in length, structure, and topic. A Transformer can read an entire question at once and relate distant words to each other, for example connecting “deadline” in “what is the deadline for the assignment mentioned earlier” back to “assignment” even if many words separate them. This makes Transformers well suited to understanding full questions rather than just matching keywords.

2. The role of attention mechanisms

Attention is what lets the model skip the filler and zero in on what actually matters. Ask "When is the CAT for BIT4133 scheduled?" and it weighs "CAT," "BIT4133," and "scheduled" heavily, while "the" and "is" barely register.

3. Three benefits over traditional RNN models

- Speed - it reads the question all at once instead of word by word
- Long-range understanding - it holds onto early words in a sentence instead of losing track of them the way RNNs tend to
- Scalability - it can train on far more student queries without the accuracy dropping off

CLASS EXERCISE 2

Given the sentence:

"The doctor informed the patient that she would recover soon."

1. How self-attention helps determine the meaning of "she"

The model checks "she" against every other word in the sentence and scores how relevant each one is. "Doctor" and "patient" are both in the running. What tips it is "recover" that's something a patient does, not a doctor so the model leans toward "patient" as the answer.

2. Words likely to receive high attention scores

- "patient" - the strongest candidate
- "recover" - the clue that points to the answer
- "doctor" - still in the running, just less likely

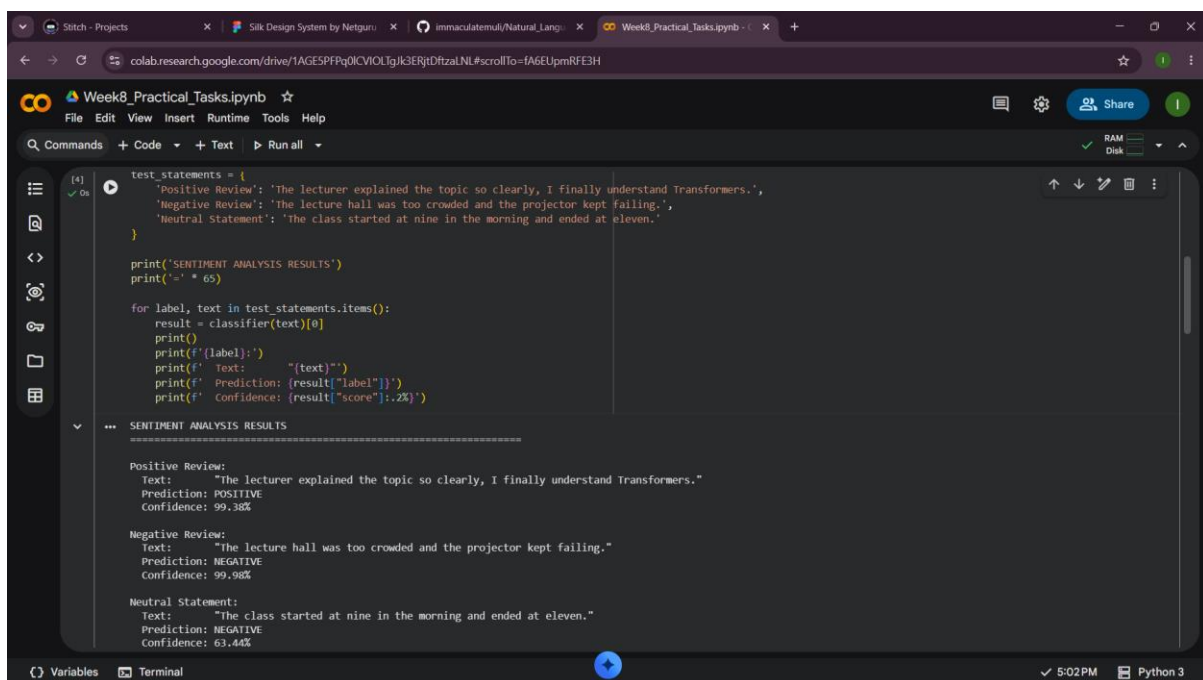
"The" and "that" barely get noticed since they don't carry much meaning on their own.

3. Why context is important in NLP

On its own, "she" could mean anyone. It only resolves once you look at what's around it. Most of language works this way pronouns, vague phrasing, things left unsaid and a model that ignores the surrounding words is going to guess wrong constantly.

PRACTICAL TASK 1

Fig 1: Hugging Face Transformers that performs sentiment analysis



```
test_statements = {
    'Positive Review': 'The lecturer explained the topic so clearly, I finally understand Transformers.',
    'Negative Review': 'The lecture hall was too crowded and the projector kept failing.',
    'Neutral Statement': 'The class started at nine in the morning and ended at eleven.'
}

print("SENTIMENT ANALYSIS RESULTS")
print("-" * 65)

for label, text in test_statements.items():
    result = classifier(text)[0]
    print()
    print(f'Label: {label}')
    print(f'Text: {text}')
    print(f'Prediction: {result["label"]}')
    print(f'Confidence: {result["score"]:.2%}')

print()
print("SENTIMENT ANALYSIS RESULTS")
print("=====")
```

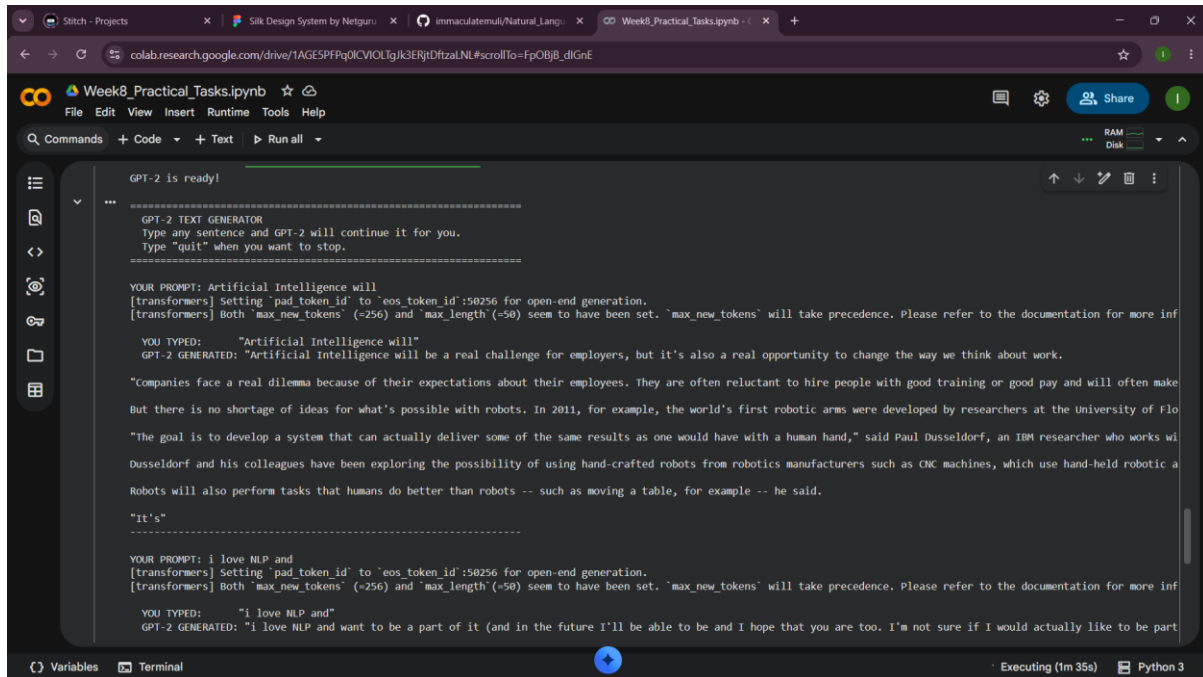
Positive Review:
Text: "The lecturer explained the topic so clearly, I finally understand Transformers."
Prediction: POSITIVE
Confidence: 99.38%

Negative Review:
Text: "The lecture hall was too crowded and the projector kept failing."
Prediction: NEGATIVE
Confidence: 99.98%

Neutral Statement:
Text: "The class started at nine in the morning and ended at eleven."
Prediction: NEGATIVE
Confidence: 63.44%

PRACTICAL TASK 2

Fig 2: Text-generation application using GPT-2



```
GPT-2 is ready!

=====
GPT-2 TEXT GENERATOR
Type any sentence and GPT-2 will continue it for you.
Type "quit" when you want to stop.
=====

YOUR PROMPT: Artificial Intelligence will
[transformers] Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.
[transformers] Both 'max_new_tokens' (~256) and 'max_length' (~50) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more info.

YOU TYPED: "Artificial Intelligence will"
GPT-2 GENERATED: "Artificial Intelligence will be a real challenge for employers, but it's also a real opportunity to change the way we think about work.

"Companies face a real dilemma because of their expectations about their employees. They are often reluctant to hire people with good training or good pay and will often make
But there is no shortage of ideas for what's possible with robots. In 2011, for example, the world's first robotic arms were developed by researchers at the University of Flo
"The goal is to develop a system that can actually deliver some of the same results as one would have with a human hand," said Paul Dusseldorf, an IBM researcher who works wi
Dusseldorf and his colleagues have been exploring the possibility of using hand-crafted robots from robotics manufacturers such as CNC machines, which use hand-held robotic a
Robots will also perform tasks that humans do better than robots -- such as moving a table, for example -- he said.

"it's"
=====

YOUR PROMPT: i love NLP and
[transformers] Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.
[transformers] Both 'max_new_tokens' (~256) and 'max_length' (~50) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more info.

YOU TYPED: "i love NLP and"
GPT-2 GENERATED: "i love NLP and want to be a part of it (and in the future i'll be able to be and i hope that you are too. I'm not sure if i would actually like to be part
```

EXTRA TASKS

Week 1: Extra tasks

Fig 1: Stemming and lemmatization

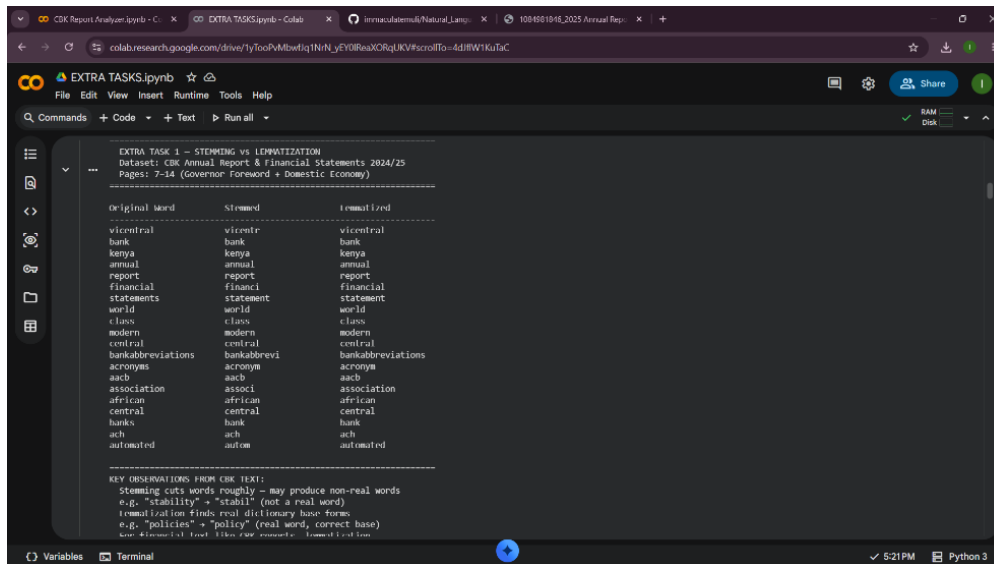


Fig 2: Word frequency and counter visualization

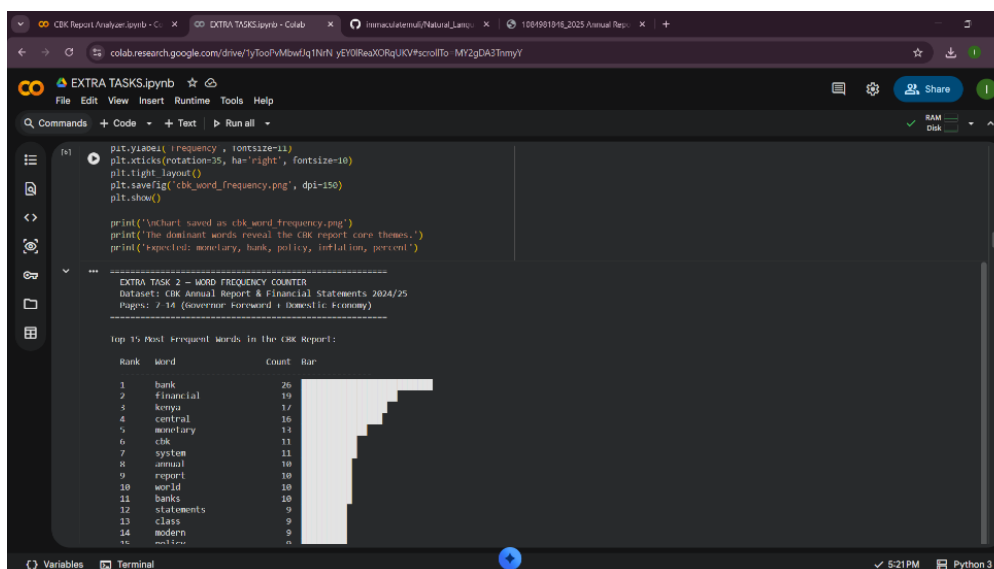
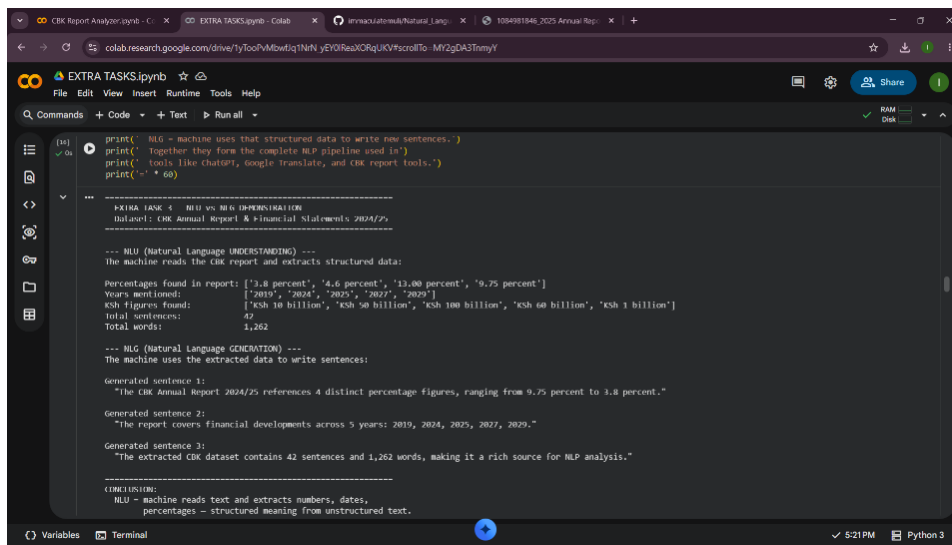


Fig 3: NLU vs NLG demonstration



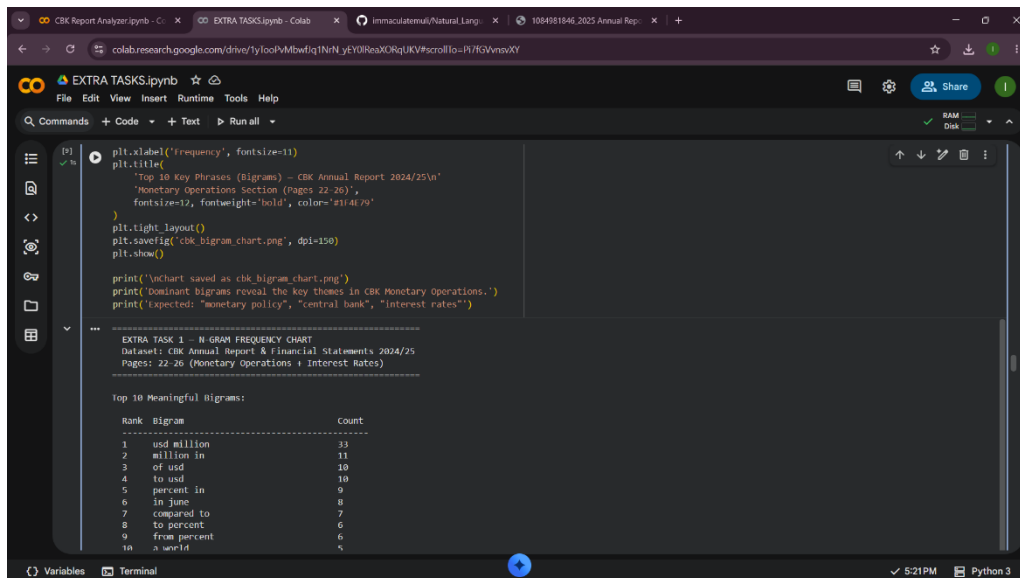
```
print(' NLG = machine uses that structured data to write new sentences. )  
print(' Together they form the complete MLP pipeline used in )  
print(' tools like ChatGPT, Google translate, and CBK report tools. )  
print(' " " * 60 )  
  
=====   
EXTRA TASK 4 NLU vs NLG DEMONSTRATION   
Dataset: CBK Annual Report & Financial Statements 2024/25   
=====   
  
--- NLU (Natural Language UNDERSTANDING) ---   
The machine reads the CBK report and extracts structured data:   
  
Percentages found in report: ['3.8 percent', '4.6 percent', '13.00 percent', '9.75 percent']   
Years mentioned: ['2019', '2024', '2025', '2022', '2023']   
KSh figures found: ['KSh 10 billion', 'KSh 50 billion', 'KSh 100 billion', 'KSh 60 billion', 'KSh 1 billion']   
Total sentences: 42   
Total words: 1,262   
  
--- NLG (Natural Language GENERATION) ---   
The machine uses the extracted data to write sentences:   
  
Generated sentence 1:   
"The CBK Annual Report 2024/25 references 4 distinct percentage figures, ranging from 9.75 percent to 3.8 percent."  
  
Generated sentence 2:   
"The report covers financial developments across 5 years: 2019, 2024, 2025, 2022, 2023."  
  
Generated sentence 3:   
"The extracted CBK dataset contains 42 sentences and 1,262 words, making it a rich source for MLP analysis."  
  
=====   
CONCLUSION:   
NLU = machine reads text and extracts numbers, dates,   
percentages - structured meaning from unstructured text.
```

What I learned

In the stemming and lemmatization extra task, I discovered that stemming roughly cuts CBK financial words "*stability*" became "*stabil*", which is not a real word while lemmatization correctly returned "*policies*" to "*policy*", a proper dictionary form. The word frequency chart visually confirmed that "*monetary*", "*bank*", and "*inflation*" dominate the CBK report, proving the document is focused on monetary poicy. The NLU versus NLG demonstration showed how machines extract percentages and years from CBK text then automatically generate summary sentences from that data exactly how tools like ChatGPT process financial documents.

Week 2: Extra tasks

Fig 1: N-gram Frequency Chart



What I learned

The N-gram frequency chart extra task extended the mandatory bigram work by filtering out pairs where both words were stopwords, leaving only meaningful economic phrases.

Visualizing the top ten bigrams as a horizontal bar chart made it immediately clear which phrases dominate the CBK Monetary Operations section "*monetary policy*" and "*central bank*" appeared most frequently

Week 3: Extra tasks

Fig 1: displaCy NER Visualization

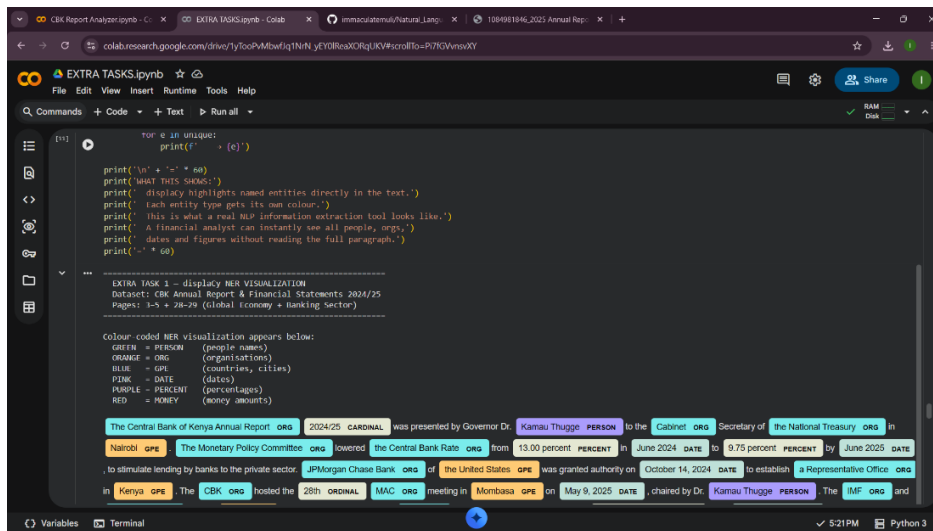
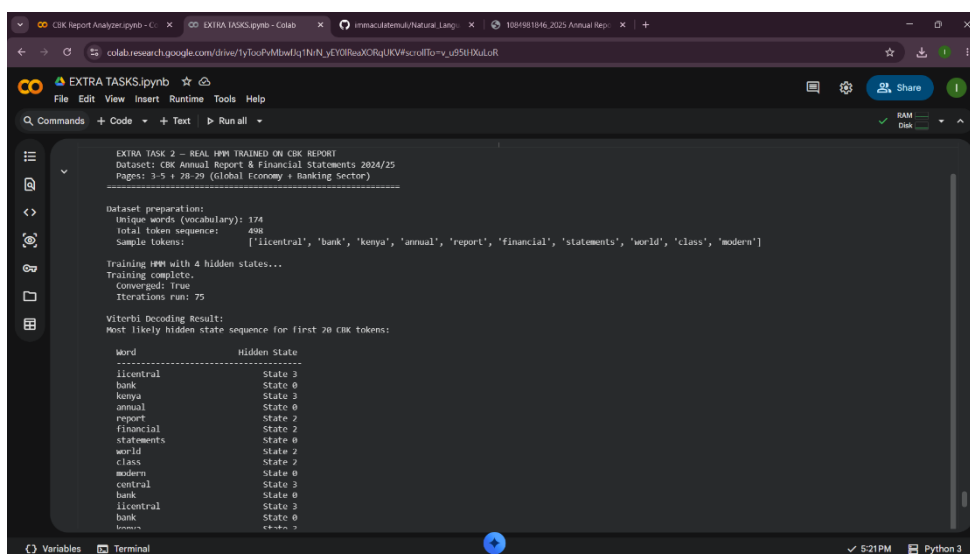


Fig 2: Real HMM trained on CBK Report data



What I learned

The displaCy extra task highlighted named entities from the CBK report using colours people, organisations, locations, and percentages each appeared in a different colour, making it easy to spot Dr. Kamau Thugge, JPMorgan Chase, Nairobi, and 9.75 percent instantly without reading the full paragraph. The hmmlearn extra task trained a real Hidden Markov Model on CBK text by converting every word into a number, teaching the model patterns in CBK financial language, then using the Viterbi algorithm to find the most likely hidden labels behind each word. This is exactly how Gmail spam detection and Google speech recognition work behind the scenes.